FULL-STACK JOB PORTAL WITH AI-BASED RESUME SCREENING AND CANDIDATE MATCHING

PROJECT REPORT

Submitted by
SAMRUDHI VISHAL DUBAL
(EC2552001011088)

Under the Guidance of

Anjana

in partial fulfillment for the award of the degree of

MASTER OF BUSINESS ADMINISTRATION
(Artificial Intelligence and Data Science)



DIRECTORATE OF ONLINE EDUCATION,
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY,
KATTANKULATHUR- 603 203

November 2026

BONAFIDE CERTIFICATE

This Project Report titled **"Full-Stack Job Portal with AI-Based Resume Screening and Candidate Matching"** is the Bonafide work of **SAMRUDHI VISHAL DUBAL (EC2552001011088)** who carried out the Project under my supervision along with the company mentor. Certified further, that to the best of my knowledge the work reported herein does not form any other internship report or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE	**SIGNATURE**
Anjana	
PROJECT GUIDE	EXAMINER

CERTIFICATE OF PROJECT

This is to certify that SAMRUDHI VISHAL DUBAL, Registration Number EC2552001011088,
underwent Project Work from [Start Date] to [End Date] under the guidance of Anjana.
This work has been carried out in partial fulfillment of the requirements for the award of the degree of **MASTER OF BUSINESS ADMINISTRATION (Artificial Intelligence and Data Science).**

Place:

Date:

SIGNATURE OF THE GUIDE	**SIGNATURE OF THE STUDENT**
Anjana	Samrudhi Vishal Dubal
	EC2552001011088

ACKNOWLEDGEMENTS

I take this opportunity to express my sincere gratitude to everyone who supported me in the successful completion of this project, titled **"Full-Stack Job Portal with AI-Based Resume Screening and Candidate Matching"**.

I am thankful to the management of **SRM Institute of Science and Technology** and the **Directorate of Online Education** for providing a flexible and well-structured learning environment that made it possible for me to pursue the Master of Business Administration programme with a specialisation in Artificial Intelligence and Data Science.

I express my deep and heartfelt gratitude to my project guide, **Anjana**, for the valuable guidance, constant encouragement and constructive feedback provided at every stage of this project. The suggestions I received helped me shape the problem statement, refine the scope of the solution and present the work in a structured manner.

I also thank all the faculty members of the Directorate of Online Education for the knowledge they shared through the programme, particularly in the areas of business analytics, machine learning and information systems, which formed the foundation of this work.

I am grateful to the open-source community behind Node.js, Express, MongoDB, React, scikit-learn and the many libraries used in this project, whose freely available tools and documentation made this work possible.

Finally, I thank my family and friends for their patience, moral support and encouragement throughout the duration of the programme and this project.

Samrudhi Vishal Dubal

EC2552001011088

TABLE OF CONTENT

CHAPTER	PARTICULARS	PAGE NO.
I	ACKNOWLEDGEMENTS	–
	ABSTRACT	1
II	INTRODUCTION	2
III	ANALYSIS AND REQUIREMENTS	10
IV	MODULE DESCRIPTION	18
V	DESIGN	26
VI	SAMPLE CODE	36
VII	OUTPUT	45
VIII	TESTING	54
IX	CONCLUSION	59
X	FUTURE ENHANCEMENTS	61
XI	REFERENCES	62
XII	APPENDIX	65

ABSTRACT

Recruiters today receive hundreds of applications for a single opening, and manual resume screening is slow, inconsistent and prone to missing qualified candidates, while commercial applicant tracking systems are costly and depend on rigid keyword filters. This project designs and develops **JobMatch AI**, a full-stack job portal built on the MERN stack (MongoDB, Express.js, React and Node.js) with an integrated, explainable **AI-based resume screening and candidate matching engine**. Candidates search jobs and apply by uploading a resume in PDF, DOCX, DOC or TXT format; employers post jobs and see applicants automatically ranked by a 0–100 match score; and administrators monitor platform statistics and moderate users and jobs. The score blends skill overlap (50%), detected through a curated skills dictionary and an alias table that treats terms such as "JS" and "JavaScript" alike, term-frequency cosine similarity between the resume and the job description (25%), and experience fit against the seniority of the job (25%); missing experience is treated neutrally and education is shown but not scored, to support fairness. An optional large-language-model layer can add a qualitative fit summary, and email notifications keep all parties informed. My contribution covered requirement analysis, system design, development of the backend, frontend and matching engine, an exploratory analysis and Random Forest classification pipeline on a 12,000-record talent-recruitment dataset, and verification through 57 automated unit and component tests, all of which pass, together with a 21-case API integration suite. In a demonstration with four sample candidates, the engine ranked a strong MERN developer first (84%) and an unrelated profile last (0%) while listing the matched and missing skills behind every score. The study shows that affordable, transparent AI can reduce screening effort, improve consistency and shorten time-to-hire, while leaving the final hiring decision with human recruiters.

CHAPTER II
INTRODUCTION

2.1 Background

Talent acquisition is a strategic function for every organisation. The quality of people an organisation hires directly influences its productivity, innovation and long-term competitiveness. At the same time, the recruitment process has changed dramatically over the last decade. Online job boards, professional networking platforms and company career pages have made it very easy for candidates to apply for jobs, and as a result the number of applications received for each opening has grown sharply.

This growth has created a screening bottleneck. Recruiters and hiring managers must read through large numbers of resumes, many of which do not meet the basic requirements of the role. Studies of recruiter behaviour commonly report that an initial resume review lasts only a few seconds, which means that relevant candidates can easily be missed and that decisions vary from one reviewer to another. For small and medium enterprises, which usually do not have dedicated HR technology teams, this problem is even more pronounced.

Artificial Intelligence (AI) and Natural Language Processing (NLP) offer a practical way to address this bottleneck. By automatically reading resumes, extracting skills and experience, and comparing them with job requirements, an AI system can produce a first-pass ranking of applicants within seconds. This allows recruiters to spend their limited time on the most promising candidates and on the human aspects of hiring, such as interviews and culture fit.

2.2 Overview of the HR Technology Industry

Human-resource technology (HR Tech) covers software that supports the employee life cycle — recruitment, onboarding, payroll, performance and learning. Recruitment technology is one of its fastest-growing segments and includes online job boards, professional networking platforms, applicant tracking systems (ATS), assessment tools and, increasingly, AI-driven screening and matching products. In India, the growth of digital hiring, the large number of graduates entering the workforce each year and the expansion of the technology and services sector have made online recruitment the default

channel for most organisations. Global ATS vendors largely serve large enterprises, while small and medium enterprises, start-ups and campus placement cells often continue to rely on email and spreadsheets because of cost and complexity.

The product developed in this project, **JobMatch AI**, is positioned for this under-served segment: an affordable, self-hostable job portal with built-in, explainable AI screening that does not require paid AI services to operate.

2.3 Significance of the Study

From a business-management perspective, the project addresses several key HR performance indicators:

- **Time-to-hire:** automated scoring and ranking reduce the time spent on initial screening from hours to minutes.
- **Cost-per-hire:** the solution is built entirely on open-source technologies and runs without paid AI services by default, making it affordable for small organisations and educational institutions.
- **Quality and consistency of hiring:** every applicant is evaluated against the same transparent criteria, reducing reviewer-to-reviewer variation.
- **Candidate experience:** applicants receive an immediate match score and automatic email updates about their application status, improving transparency.
- **Data-driven decision making:** the platform captures structured data on jobs, applicants and outcomes that can be analysed to improve future hiring.

The project also has academic significance for the specialisation in Artificial Intelligence and Data Science, as it combines information-retrieval techniques (tokenisation, term-frequency vectors and cosine similarity), rule-based information extraction, optional use of large language models, and supervised machine learning (Random Forest classification) within a single, working business application.

2.4 Review of Literature

Research on automated recruitment draws on information retrieval, recommender systems, machine learning and, more recently, the study of fairness in algorithmic decision making. The studies most relevant to this project are summarised below.

Job recommender systems. Siting et al. (2012) surveyed job recommender systems and classified them into content-based, collaborative-filtering, knowledge-based and hybrid approaches, observing that recruitment differs from product recommendation because both the candidate and the employer must be satisfied. Malinowski et al. (2006) formalised this as a bilateral matching problem in which recommendations must reflect the preferences of both people and jobs. Paparrizos, Cambazoglu and Gionis (2011) trained machine-learning models on historical job transitions to recommend jobs, and Kenthapadi, Le and Venkataraman (2017) described the practical challenges of operating large-scale personalised job recommendation at LinkedIn, including relevance, scale and cold-start problems.

Machine learning for candidate screening. Faliagka et al. (2012) applied machine-learning algorithms to an online recruitment system, extracting features from applicant profiles to predict a ranking that agreed closely with human recruiters. Roy, Chowdhary and Bhatia (2020) proposed a resume recommendation system that classifies resumes into job categories and uses content-based similarity to recommend the closest matches, showing that text vectorisation combined with classifiers can substantially reduce screening effort.

Text representation and similarity. The vector space model and term-weighting schemes described by Salton and Buckley (1988) and Manning, Raghavan and Schütze (2008) remain the foundation of document similarity: documents are represented as term vectors and compared using cosine similarity. More recent transformer models such as BERT (Devlin et al., 2019) capture contextual meaning more richly, but require greater computing resources and are harder to explain to end users.

Fairness and transparency in algorithmic hiring. Raghavan et al. (2020) analysed the claims and practices of commercial pre-employment assessment vendors and highlighted the lack of transparency about how models are built and validated for bias. A widely reported example is an experimental AI recruiting tool that was abandoned after it was found to disadvantage women because it had learned from historically male-dominated hiring data (Dastin, 2018). In India, the Digital Personal Data Protection Act, 2023 adds legal obligations for the handling of personal data such as resumes.

2.4.1 Research Gap

The literature shows that (i) learned models need large volumes of labelled historical hiring data, which small organisations do not have, and can reproduce historical bias; (ii) simple keyword filters are transparent but miss synonyms and abbreviations; and (iii) most studies evaluate algorithms in isolation rather than as part of a complete, usable recruitment workflow. There is a need for an affordable, explainable and synonym-aware screening approach that works without training data, treats missing information fairly and is integrated into an end-to-end job portal. This project addresses that gap.

2.5 Problem Statement

Organisations that recruit through online channels face the following problems:

1. Manual screening of a large volume of resumes is slow, repetitive and expensive.
2. Screening decisions are inconsistent because different reviewers apply different, often implicit, criteria.
3. Simple keyword filters used by many existing systems reject qualified candidates who use synonyms or abbreviations (for example "JS" instead of "JavaScript" or "k8s" instead of "Kubernetes").
4. Most automated tools behave as "black boxes" and do not explain why a candidate was ranked high or low.
5. Candidates rarely receive timely feedback on the status of their applications.

The problem addressed by this project is therefore: **to design and develop a secure, role-based, full-stack job portal that automatically screens every resume against the requirements of the job, produces an explainable match score, ranks applicants for employers, and keeps all stakeholders informed — without depending on costly proprietary software.**

2.6 Objectives of the Study

2.6.1 Primary Objective

To design and develop a full-stack job portal with AI-based resume screening and candidate matching.

2.6.2 Secondary Objectives

1. To develop a responsive web application where employers can post jobs and candidates can search for and apply to them.
2. To implement secure authentication and role-based access control for candidates, employers and administrators.
3. To extract text from resumes uploaded in PDF, DOCX, DOC and TXT formats.
4. To design an AI-based matching engine that scores each application from 0 to 100 using skill overlap, text similarity and experience fit, and explains the score through matched and missing skills.
5. To present employers with a ranked list of applicants and allow them to manage the hiring pipeline (applied, shortlisted, rejected, hired).
6. To provide an administrative dashboard for platform-wide statistics and moderation.
7. To send automatic email notifications on application submission and status changes.
8. To analyse a recruitment dataset and build a machine-learning model that predicts candidate suitability.
9. To verify the system through automated unit, integration and component tests.

2.7 Scope of the Project

The scope of the project includes the complete design, development and testing of the JobMatch AI web application, consisting of a React single-page frontend, a Node.js/Express REST API, a MongoDB database and an AI screening engine that runs locally on the server. The system covers the full recruitment cycle from job posting to final hiring status. It is suitable for small and medium enterprises, campus placement cells, staffing agencies and internal hiring portals of larger organisations. The supporting data-analysis work covers exploratory data analysis and a Random Forest classifier on a talent-recruitment dataset.

2.8 Limitations

- Skill detection is based on a curated dictionary and alias table; skills not present in the dictionary are only recognised if the candidate lists them explicitly in the profile or if the employer lists them as required skills.

- Years of experience are extracted with pattern matching (for example "5 years of experience"); experience described only through dates of employment is not calculated.
- Scanned (image-only) PDF resumes are not supported because the system does not perform optical character recognition (OCR).
- The text-similarity measure is lexical (term-frequency based); it does not fully capture meaning in the way modern embedding models do. The optional LLM layer partly addresses this but requires an API key.
- The machine-learning analysis was carried out on a publicly available, largely synthetic dataset and is presented as a study of methodology rather than a production model.

2.9 Research Methodology

2.9.1 Research Design

The study follows an **applied, design-oriented research design**: a working information system (the artefact) is designed, built and evaluated to solve an identified business problem. The evaluation combines automated testing of the software with quantitative validation of the matching engine on sample resumes, supported by an exploratory and predictive analysis of secondary recruitment data.

2.9.2 Sources of Data

- **Secondary data:** a publicly available talent-recruitment dataset of 12,000 candidate records and 26 attributes, used for exploratory data analysis and machine-learning modelling; published research papers, books and official documentation used for the literature review and system design.
- **Validation data:** job postings and sample resumes prepared to represent strong, partial and weak matches, used to validate the ranking produced by the matching engine.
- **Requirement inputs:** study of existing job portals and applicant tracking systems, and the needs of candidates, employers and administrators.

2.9.3 Tools and Techniques of Analysis

- Descriptive statistics, distribution plots and correlation analysis (pandas, Matplotlib, Seaborn).

- Supervised classification with a Random Forest model, evaluated using a stratified 80:20 train–test split (scikit-learn).
- Information-retrieval techniques — tokenisation, stop-word removal, term-frequency vectors and cosine similarity — for resume–job matching.
- Rule-based information extraction with regular expressions for experience and education.
- Automated unit, integration and component testing (Jest, Supertest, Vitest, React Testing Library).

2.9.4 Development Approach

The software was built using an **iterative and incremental (Agile)** approach. The work was divided into short cycles, each delivering a usable increment of functionality that was tested before moving on:

1. **Requirement analysis** — study of existing job portals and ATS products, identification of stakeholders and their needs.
2. **Design** — definition of the system architecture, database schema, REST API and user-interface flows.
3. **Core development** — authentication, job management, resume upload and parsing.
4. **AI engine development** — skills dictionary, alias resolution, text-similarity scoring, experience and education extraction, and score blending.
5. **Enhancement** — admin dashboard, email notifications and the optional LLM-based semantic layer.
6. **Data analysis** — exploratory analysis and Random Forest modelling of the recruitment dataset in Python (Google Colab).
7. **Testing and validation** — automated unit, integration and component tests and manual end-to-end testing.

2.10 Organisation of the Report

Chapter III analyses the existing system, specifies the requirements of the proposed system and presents the analysis and interpretation of the recruitment dataset. Chapter IV describes each module. Chapter V presents the system design with architecture, data-flow, UML and database diagrams. Chapter VI explains important parts

of the source code. Chapter VII presents the output screens and sample results, and Chapter VIII describes the testing performed. Chapters IX and X give the conclusion, suggestions and future enhancements, followed by the references and appendices.

CHAPTER III
ANALYSIS AND REQUIREMENTS

3.1 Existing System

In most small and medium organisations, recruitment is still handled through a combination of job-board postings, email inboxes and spreadsheets. Resumes arrive as email attachments or are downloaded from job boards, and HR staff open each file individually to judge whether the applicant is suitable. Larger organisations use commercial Applicant Tracking Systems, which store applications centrally and usually provide keyword-based filtering.

3.2 Limitations of the Existing System

- **Time-consuming manual review:** every resume has to be opened and read, which does not scale when there are hundreds of applicants.
- **Subjectivity and inconsistency:** two reviewers may evaluate the same resume differently; fatigue also affects judgement.
- **Rigid keyword matching:** traditional filters look for exact words and miss synonyms and abbreviations, rejecting qualified candidates.
- **Lack of explainability:** many automated tools give a pass/fail outcome without stating which requirements were met or missed.
- **High cost:** commercial ATS products are licensed per recruiter or per job, which is a barrier for small businesses and institutions.
- **Poor candidate communication:** applicants often receive no acknowledgement or status update.
- **Scattered data:** information spread across emails and spreadsheets is hard to analyse for improving hiring decisions.

3.3 Proposed System

The proposed system, **JobMatch AI**, is a centralised web-based job portal with a built-in AI screening engine. Each application is scored automatically at the moment it is submitted, and employers see their applicants already ranked by suitability, with a breakdown that explains the score.

3.3.1 Advantages of the Proposed System

- Instant, automated screening and ranking of every applicant.
- Explainable scores showing skill match, text similarity, experience fit, matched skills and missing skills.
- Synonym-aware skill matching through an alias table (for example js → javascript, k8s → kubernetes, rest api → rest).
- Fair treatment of incomplete information — if the resume does not mention years of experience, the candidate is not penalised; the weight is redistributed to the other signals.
- Education level is shown for information but not used in the score, to avoid disadvantaging candidates with non-traditional backgrounds.
- Works fully offline with no paid API; an LLM-based semantic score can be enabled optionally.
- Role-based dashboards for candidates, employers and administrators, with automatic email notifications.

3.4 Stakeholder Analysis

Table 3.1 Stakeholders and their needs

Stakeholder	Role in the System	Key Needs
Candidate (job seeker)	Searches jobs, applies with a resume, tracks applications	Easy search, simple application, quick and transparent feedback
Employer / Recruiter	Posts jobs, reviews and manages applicants	Less screening effort, ranked shortlist, clear reasons for each score
Administrator	Monitors and moderates the platform	Platform statistics, control over users and job postings
Organisation management	Sponsors the hiring process	Lower time-to-hire and cost-per-hire, better quality of hire

3.5 Functional Requirements

Table 3.2 Functional requirements

ID	Requirement
FR1	Users shall be able to register as a candidate or employer and log in with email and password; administrators cannot self-register.
FR2	The system shall restrict every protected operation according to the user's role and block deactivated accounts.

ID	Requirement
FR3	Users shall be able to view and update their profile (name, title, company, skills, bio).
FR4	Employers shall be able to create, edit, close, reopen and delete job postings with title, company, description, location, job type, experience level, salary range and required skills.
FR5	Anyone shall be able to search open jobs by keyword and filter them by location, job type, experience level and skill.
FR6	Candidates shall be able to apply to an open job by uploading a resume (PDF, DOCX, DOC or TXT, up to 5 MB) with an optional cover letter; duplicate applications shall be rejected.
FR7	The system shall extract text from the resume and compute a 0–100 match score with skill match, text similarity, experience fit, education level, matched skills and missing skills.
FR8	Employers shall see applicants for their own jobs sorted by match score and be able to change each applicant's status (applied, shortlisted, rejected, hired).
FR9	Candidates shall be able to view all their applications with match score and status.
FR10	The system shall email the employer on a new application and the candidate on submission and on every status change.
FR11	Administrators shall be able to view platform statistics, search and filter users, activate/deactivate/delete users, and close, reopen or delete any job, and view all applications.
FR12	Optionally, when an LLM API key is configured, the system shall blend an AI-generated fit score and summary into the final score.

3.6 Non-Functional Requirements

Table 3.3 Non-functional requirements

Category	Requirement
Security	Passwords hashed with bcrypt; stateless JWT authentication; role-based authorisation; uploaded resumes are not served publicly because they contain personal data.
Performance	The local matching engine runs in memory without external calls, scoring an application in milliseconds; job lists are paginated (maximum 50 per page).
Reliability	Failures in email delivery or the optional LLM layer never block an application; orphaned uploaded files are removed when an application fails.
Usability	Clean, responsive interface built with Tailwind CSS; colour-coded match badges (green ≥ 75%, amber 50–74%, red < 50%).
Maintainability	Layered code structure (routes, controllers, models, middleware, utilities); automated test suites for backend and frontend.
Portability	Runs on any operating system with Node.js 18+ and MongoDB (local or MongoDB Atlas); configuration through environment variables.
Scalability	Stateless API servers can be scaled horizontally; MongoDB indexes support text search and prevent duplicate applications.

Category	Requirement
Fairness & Privacy	Education level is not weighted in the score; unknown experience is scored neutrally; personal data is accessible only to authorised users.

3.7 Hardware and Software Requirements

Table 3.4 Hardware requirements (development / deployment)

Component	Minimum Specification
Processor	Dual-core 2.0 GHz or higher
RAM	4 GB (8 GB recommended for development)
Storage	2 GB free disk space plus space for uploaded resumes
Network	Internet connection for package installation, email (SMTP) and optional LLM API

Table 3.5 Software requirements

Layer	Technology
Operating system	Windows 10/11, macOS or Linux
Runtime	Node.js 18 or later
Frontend	React 18, React Router 6, Vite 5, Tailwind CSS 3, Axios
Backend	Express 4, Mongoose 8, JSON Web Token, bcryptjs, Multer, pdf-parse, mammoth, Nodemailer
Database	MongoDB 6+ (local) or MongoDB Atlas
AI (optional)	Anthropic Claude API via @anthropic-ai/sdk
Data analysis	Python 3, pandas, NumPy, Matplotlib, Seaborn, scikit-learn (Google Colab)
Testing	Jest, Supertest, mongodb-memory-server, Vitest, React Testing Library
Tools	Visual Studio Code, Git and GitHub, web browser (Chrome/Edge/Firefox)

3.8 Feasibility Analysis

3.8.1 Technical Feasibility

All technologies used are mature, widely adopted and well documented. The MERN stack uses JavaScript across the frontend and backend, which simplifies development. The AI engine is implemented with standard information-retrieval techniques that run efficiently on ordinary hardware without GPUs. The project is therefore technically feasible.

3.8.2 Economic Feasibility

Every component of the stack is open-source and free of licence fees. The default matching engine does not call any paid AI service, so there is no per-application cost. The application can be hosted on low-cost cloud infrastructure, and MongoDB Atlas offers a free tier suitable for pilots. Against this minimal cost, the organisation saves a significant amount of recruiter time per job opening. The project is economically feasible.

3.8.3 Operational Feasibility

The interface is simple and follows familiar job-portal conventions, so candidates and recruiters need little or no training. The ranked applicant list fits naturally into the existing shortlisting workflow and the status pipeline mirrors typical hiring stages. The project is operationally feasible.

3.8.4 Legal and Ethical Feasibility

Resumes contain personal data. The system restricts access to resumes to the candidate, the owning employer and administrators, does not expose uploaded files publicly, and stores passwords only as salted hashes, which is in line with the principles of India's Digital Personal Data Protection Act, 2023. To reduce bias, the score is based only on job-relevant signals (skills, content relevance and experience); education is displayed but not scored, and the final hiring decision always remains with a human recruiter.

3.9 Data Analysis and Interpretation of Recruitment Data

To understand the factors that influence candidate suitability, a talent-recruitment dataset was analysed in Python using Google Colab. The dataset contains **12,000 candidate records and 26 attributes**, including demographic information (gender, age, location), education (highest degree, specialisation, CGPA), experience and role (years of experience, current role, preferred job title, industry, career level), compensation (current and expected salary), assessment scores (communication, leadership, problem solving, resume score), activity (job applications count, interview attempts), recruiter rating and the target variable **Target_Category**.

Table 3.6 Summary of the recruitment dataset

Property	Value
Records × attributes	12,000 × 26

Property	Value
Missing values	None (0 in every column)
Duplicate records	0
Numerical features (12)	Age, CGPA, Experience_Years, Current_Salary_CNY_K, Expected_Salary_CNY_K, Communication_Score, Leadership_Score, Problem_Solving_Score, Resume_Score, Job_Applications_Count, Interview_Attempts, Recruiter_Rating
Categorical features (12)	Gender, Location, Highest_Degree, Specialization, Current_Role, Preferred_Job_Title, Industry, Company_Name, Preferred_Work_Mode, Career_Level, Technical_Skills, Certifications
Target classes	Highly Suitable (4,114), Less Suitable (3,999), Moderately Suitable (3,887)
Train / test split	9,600 / 2,400 records (80:20, stratified)

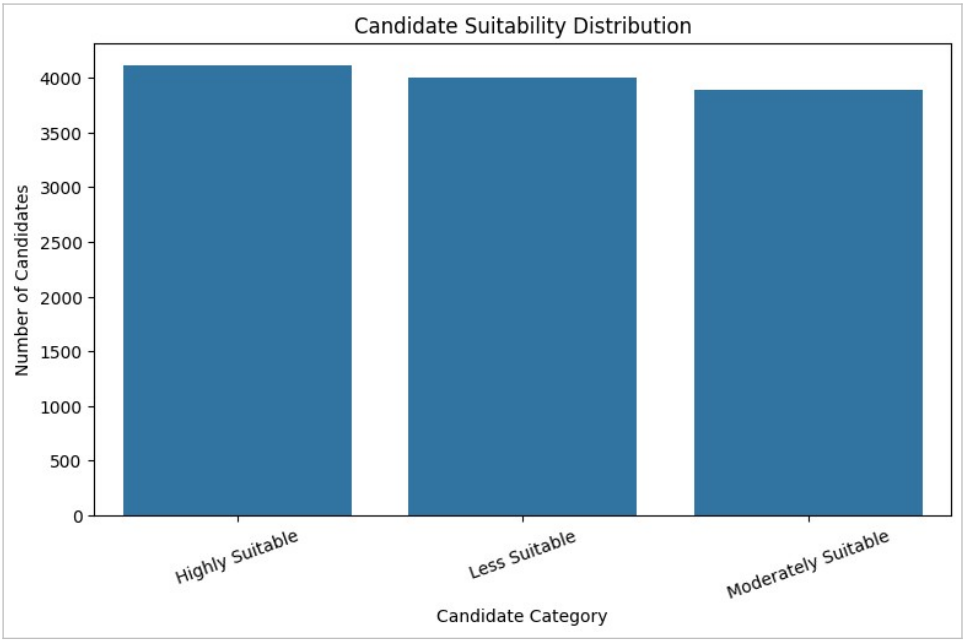


Fig. 3.1 Distribution of candidates across suitability categories

Fig. 3.1 shows that the three target classes are almost evenly balanced, so the model does not need re-sampling; nevertheless, class weighting was enabled as a safeguard. The histograms of the numerical attributes (Fig. 3.2) show broadly uniform distributions across their ranges, and the correlation matrix (Fig. 3.3) shows that the numerical attributes are almost independent of each other, with the single exception of current and expected salary, which are strongly correlated ($r = 0.97$). This indicates that expected salary adds little information beyond current salary, and that suitability is likely

to depend on combinations of attributes rather than on any single factor — a situation well suited to ensemble tree models such as Random Forest.

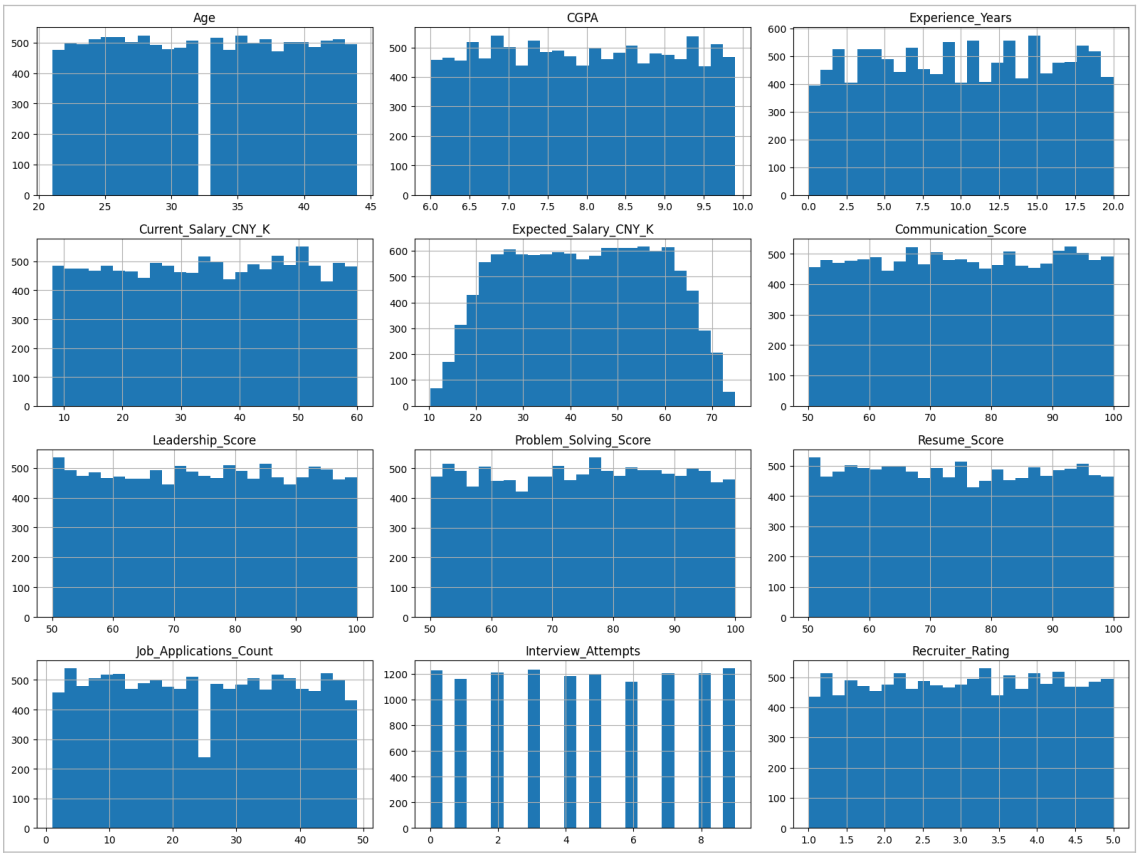


Fig. 3.2 Distribution of numerical attributes in the recruitment dataset

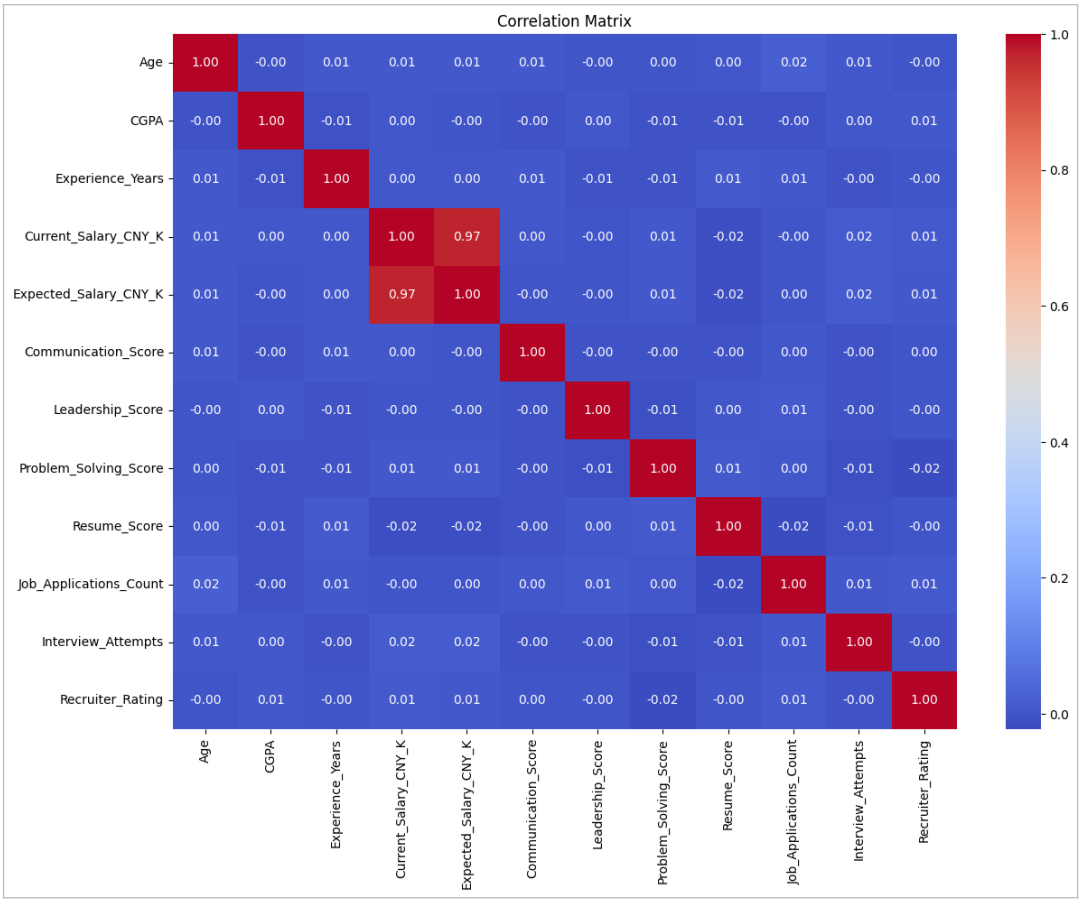


Fig. 3.3 Correlation matrix of numerical attributes

3.9.1 Findings from the Data Analysis

1. The dataset is complete and clean — no missing values and no duplicate records — so it can be modelled without imputation of real gaps.
2. The three suitability classes are nearly balanced (34.3%, 33.3% and 32.4%), so overall accuracy is a meaningful evaluation measure and re-sampling is not needed.
3. Numerical attributes such as CGPA, experience and assessment scores are spread almost uniformly and are nearly uncorrelated with each other; no single attribute dominates.
4. Current and expected salary are strongly correlated (r = 0.97); one of the two can be dropped without much loss of information.
5. Because suitability depends on combinations of attributes, a non-linear ensemble model such as Random Forest, combined with one-hot encoding of categorical attributes, is an appropriate modelling choice.

CHAPTER IV
MODULE DESCRIPTION

The JobMatch AI system is divided into the following functional modules. Each module is implemented as a set of Express routes and controllers on the backend and corresponding React pages and components on the frontend.

1. User Authentication and Authorisation Module
2. User Profile Module
3. Job Management Module
4. Job Search Module
5. Resume Upload and Parsing Module
6. AI Resume Screening and Matching Module
7. Optional LLM Semantic Scoring Module
8. Applicant Ranking and Pipeline Module
9. Email Notification Module
10. Administration Module
11. Recruitment Analytics (Machine Learning) Module

4.1 User Authentication and Authorisation Module

This module manages registration, login and access control. During registration a user chooses the role of candidate or employer; administrator accounts can only be created by a seeding script, so no one can register as an administrator through the public API. Passwords are hashed with bcrypt (10 salt rounds) before being stored. On successful login the server issues a signed JSON Web Token (JWT), which the frontend stores and attaches to every request. The `protect` middleware verifies the token and checks that the account still exists and is active, and the `authorize` middleware checks the user's role for each protected route.

Table 4.1 Authentication module summary

Aspect	Description
Purpose	Identify users securely and restrict actions according to their role.
Inputs	Name, email, password and role (registration); email and password (login); JWT on subsequent requests.

Aspect	Description
Processing	Validate input, hash password, check for duplicate email, verify credentials, sign and verify JWT, check isActive flag and role.
Outputs	JWT token and safe user object (password removed); HTTP 401/403 errors for invalid or unauthorised requests.
Interacts with	All other modules (every protected route uses protect/authorize); frontend AuthContext and PrivateRoute.

4.2 User Profile Module

Every user can view and update their own profile. Candidates maintain a headline, a list of skills and a short bio; employers maintain their company name. The skills declared on a candidate's profile are combined with the skills detected in the resume during matching, so candidates can make sure their key skills are always considered.

Table 4.2 Profile module summary

Aspect	Description
Purpose	Maintain personal and professional details of users.
Inputs	Name, title/headline, company, comma-separated skills, bio.
Processing	Authenticated update of the current user's document (PUT /api/users/me).
Outputs	Updated user profile reflected immediately in the interface.
Interacts with	Authentication module; AI Matching module (declared skills).

4.3 Job Management Module

Employers use this module to create and maintain job postings. A job contains a title, company, description, location, job type (full-time, part-time, contract or internship), experience level (entry, mid, senior or lead), an optional salary range and a list of required skills. Required skills are normalised to lowercase on save. Only the employer who created a job can edit, close, reopen or delete it; deleting a job also deletes its applications. Each job keeps a running count of applications.

Table 4.3 Job management module summary

Aspect	Description
Purpose	Allow employers to publish and manage job openings.
Inputs	Job details entered in the Post Job / Edit Job forms.
Processing	Validation of mandatory fields, normalisation of skills, ownership checks for update and delete, cascading deletion of applications.
Outputs	Job documents in MongoDB; "My Postings" list with applicant counts.

Aspect	Description
Interacts with	Authentication, Job Search, AI Matching (required skills, description, experience level), Administration.

4.4 Job Search Module

This public module lists open jobs and lets visitors search and filter them. Keyword search uses a MongoDB text index on job title, description and company. Results can be filtered by location (case-insensitive partial match), job type, experience level and a specific required skill, and are returned newest first with server-side pagination.

Table 4.4 Job search module summary

Aspect	Description
Purpose	Help candidates discover relevant openings.
Inputs	Search keyword, location, job type, experience level, skill, page number.
Processing	Build a MongoDB query on open jobs, apply text search and filters, paginate (default 10, max 50 per page).
Outputs	List of job cards with title, company, location, type and up to six skills; job detail page.
Interacts with	Job Management module; Resume Upload module (apply from job details).

4.5 Resume Upload and Parsing Module

When a candidate applies, the resume is uploaded with Multer. The file name is sanitised and made unique, only the extensions .pdf, .docx, .doc and .txt are accepted, and the size is limited to 5 MB. The parser then extracts plain text: PDF files are processed with pdf-parse, DOCX files with mammoth, and TXT/DOC files are read as text. If parsing fails, the uploaded file is deleted and the candidate receives a clear error message.

Table 4.5 Resume parsing module summary

Aspect	Description
Purpose	Convert uploaded resumes into plain text for analysis.
Inputs	Resume file (multipart/form-data) and optional cover letter.
Processing	File-type and size validation, safe storage, text extraction by format, clean-up on failure.
Outputs	Extracted resume text stored with the application.
Interacts with	AI Matching module; Applicant Ranking module.

4.6 AI Resume Screening and Matching Module

This is the core intelligent module of the project (implemented in `aiMatcher.js`, `skillsDictionary.js`, `skillAliases.js` and `experienceParser.js`). For every application it computes three independent signals and blends them into a single score from 0 to 100.

4.6.1 Signal 1 — Skill Match (weight 50%)

The job's required skills are converted to a canonical form using an alias table (for example "react" → "react.js", "node" → "node.js", "rest api" → "rest"). Skills in the resume are detected using a curated dictionary of more than 150 technologies and competencies, grouped into languages, frontend, backend, databases, cloud/DevOps, data/AI, mobile, testing, project management, design and security. Detection uses word-boundary matching so that, for example, "java" is not detected inside "javascript". The detected skills are merged with the skills declared on the candidate's profile, and the skill match is calculated as:

$$\text{Skill Match \%} = (\text{number of required skills found} \div \text{total required skills}) \times 100$$

4.6.2 Signal 2 — Text Similarity (weight 25%)

The resume text and the job text (title, description and required skills) are converted to lowercase, tokenised and filtered to remove common stop words. A term-frequency (TF) vector is built for each, and the cosine of the angle between the two vectors is calculated. This rewards resumes whose overall content is topically aligned with the job even when exact skill keywords are not listed.

$$\text{Similarity} = \Sigma\,(a_i \times b_i) \div (\sqrt{\Sigma}\, a_i^2 \times \sqrt{\Sigma}\, b_i^2), \quad \text{Text Similarity \%} = \text{Similarity} \times 100$$

4.6.3 Signal 3 — Experience Fit (weight 25%)

Regular expressions detect statements such as "6 years of experience", "experience of 3 years" or "4+ years in"; the largest value found is taken as the candidate's experience. It is compared with the minimum typically expected for the job's level:

Table 4.6 Minimum years of experience by job level

Experience Level	Minimum Years	Experience Fit
Entry	0	100% for any candidate
Mid	2	100% if years ≥ 2, otherwise (years ÷ 2) × 100

Experience Level	Minimum Years	Experience Fit
Senior	5	100% if years ≥ 5, otherwise (years ÷ 5) × 100
Lead	8	100% if years ≥ 8, otherwise (years ÷ 8) × 100

4.6.4 Score Blending

$$Match\ Score = round(\ 0.50 \times Skill\ Match + 0.25 \times Text\ Similarity + 0.25 \times Experience\ Fit\ )$$

If the resume does not mention years of experience, the candidate is not penalised. Instead the experience weight is redistributed proportionally over the other two signals, giving weights of approximately 0.667 for skill match and 0.333 for text similarity. The engine also detects the highest education level mentioned (PhD, Master's, Bachelor's, Associate or High School); this is displayed to employers for information only and is not part of the score.

Table 4.7 AI matching module summary

Aspect	Description
Purpose	Automatically screen and score each resume against the job.
Inputs	Resume text, job (title, description, required skills, experience level), candidate's declared skills.
Processing	Alias resolution, dictionary-based skill extraction, tokenisation and stop-word removal, TF cosine similarity, experience and education extraction, weighted blending.
Outputs	Match score (0–100), skill match %, text similarity %, experience fit %, years of experience, education level, matched skills, missing skills, extracted skills.
Interacts with	Resume Parsing, Applicant Ranking, Optional LLM module, Email Notification (score in emails).

4.7 Optional LLM Semantic Scoring Module

When an Anthropic API key is configured, the system additionally asks a Claude large language model, acting as an expert technical recruiter, to rate the candidate's overall fit from 0 to 100 and to give a summary of up to 40 words. The resume and job description are truncated to 6,000 characters each. The final score is then a blend of 70% local score and 30% semantic score, and the summary is shown to the employer. The module never throws errors: if the key is not configured, the response cannot be parsed or

the service is unavailable, it simply returns no result and the local score is used. By default the module is disabled, so the system has no external dependency.

Final Score = round(0.7 × Local Score + 0.3 × Semantic Score) (only when the LLM layer is enabled)

4.8 Applicant Ranking and Pipeline Module

Employers open the "Applicants" page of any of their jobs to see all applicants sorted by match score (highest first). Each applicant card shows a colour-coded match badge and can be expanded to show the full AI breakdown: skill match with matched and missing skills, resume/description similarity, experience fit and years mentioned, detected education, the optional AI summary and the cover letter. The employer moves each applicant through the pipeline by selecting a status — applied, shortlisted, rejected or hired. Candidates see the same score and current status for each of their applications on the "My Applications" page.

Table 4.8 Applicant ranking module summary

Aspect	Description
Purpose	Present a prioritised, explainable shortlist and manage hiring stages.
Inputs	Job ID (employer view), status changes from the employer.
Processing	Ownership check, query applications sorted by matchScore descending, validate and update status.
Outputs	Ranked applicant list with AI details; updated statuses; candidate application list.
Interacts with	AI Matching, Email Notification, Authentication.

4.9 Email Notification Module

The mailer uses Nodemailer with SMTP credentials when they are configured; otherwise it logs every email to the server console so that the application works out of the box in development. Three notifications are sent: to the employer when a new candidate applies (including the match score), to the candidate when the application is submitted, and to the candidate whenever the application status changes. Email failures are logged and never interrupt the user's request.

4.10 Administration Module

The administrator dashboard has four tabs. **Overview** shows platform statistics: total users, candidates, employers, total and open jobs, total applications, average AI match score and number of shortlisted applications. **Users** lets the administrator search users by name or email, filter by role, activate or deactivate accounts and delete users (deleting a user also removes the user's jobs and applications). **Jobs** lets the administrator force-close, reopen or delete any job. **Applications** lists every application on the platform.

Table 4.9 Administration module summary

Aspect	Description
Purpose	Monitor platform health and moderate content and users.
Inputs	Admin actions: search terms, role filter, activate/deactivate, delete, job status changes.
Processing	Aggregation queries for statistics, user and job updates with cascading clean-up.
Outputs	Statistics cards, user/job/application tables, updated records.
Interacts with	All data collections; Authentication (deactivated users are blocked immediately).

4.11 Recruitment Analytics (Machine Learning) Module

This module, developed as a Python notebook, complements the web application with a data-science study of candidate suitability. Its pipeline consists of the following stages:

1. Load the recruitment dataset and inspect its shape, columns, missing values and duplicates.
2. Exploratory data analysis: target distribution, histograms of numerical attributes and correlation matrix.
3. Remove the identifier (Candidate_ID) and separate the target (Target_Category).
4. Stratified 80:20 train/test split (9,600 training and 2,400 testing records).
5. Pre-processing with a ColumnTransformer: median imputation and standard scaling for numerical features; most-frequent imputation and one-hot encoding for categorical features.
6. Train a Random Forest classifier with 300 trees and balanced class weights.

7. Evaluate with accuracy, a classification report (precision, recall, F1-score) and a confusion matrix; rank features by importance.
8. Predict the suitability category and class probabilities for a new candidate, and demonstrate a rule-based job-matching function (experience, degree, industry, job title and work mode).
9. Save the trained pipeline with joblib for reuse.

This module demonstrates how historical hiring data could be used in future to learn suitability patterns directly from outcomes, complementing the transparent rule-based scoring used in the web application.

CHAPTER V
DESIGN

5.1 System Architecture

JobMatch AI follows a three-tier client–server architecture. The **presentation layer** is a React single-page application that communicates with the server only through a REST API. The **application layer** is a Node.js/Express server containing the middleware, route controllers, the AI screening engine and the mailer. The **data layer** consists of a MongoDB database accessed through Mongoose and a private file store for uploaded resumes.

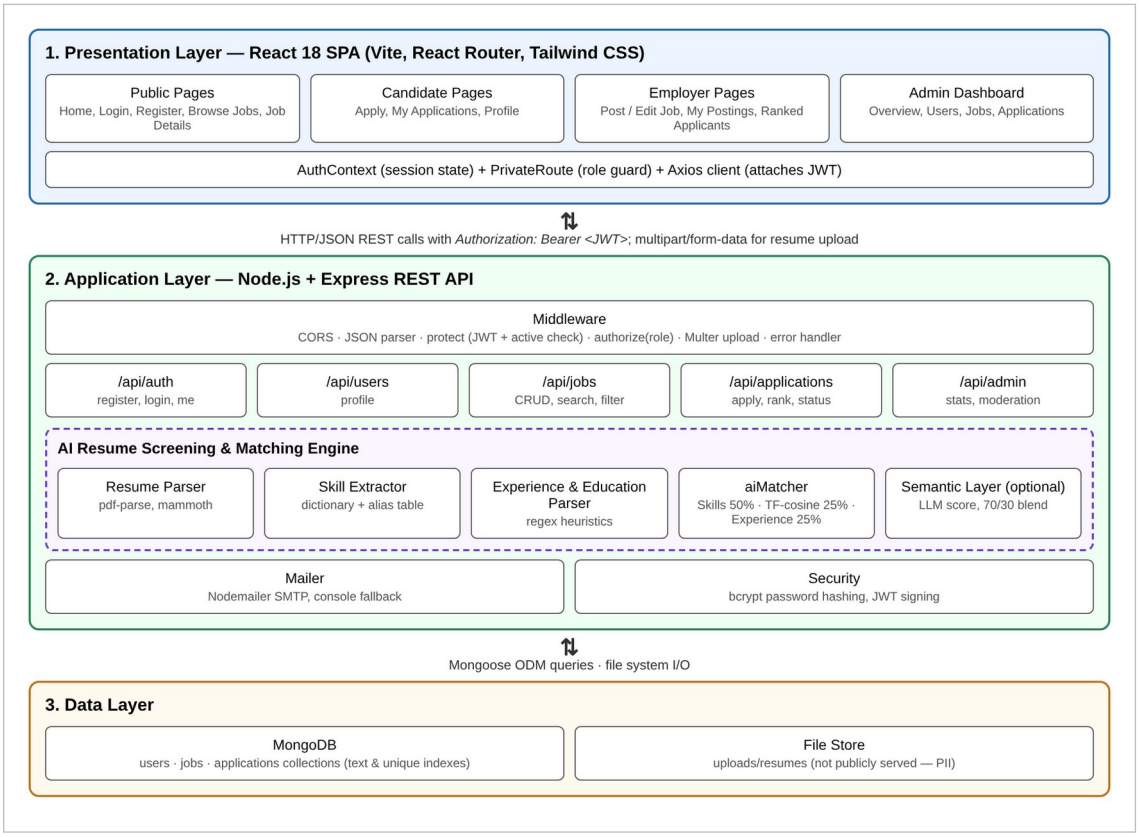


Fig. 5.1 Three-tier system architecture of JobMatch AI

5.2 Data Flow Diagram (Level 0)

The context-level data flow diagram shows the system as a single process interacting with its three external entities and its three data stores.

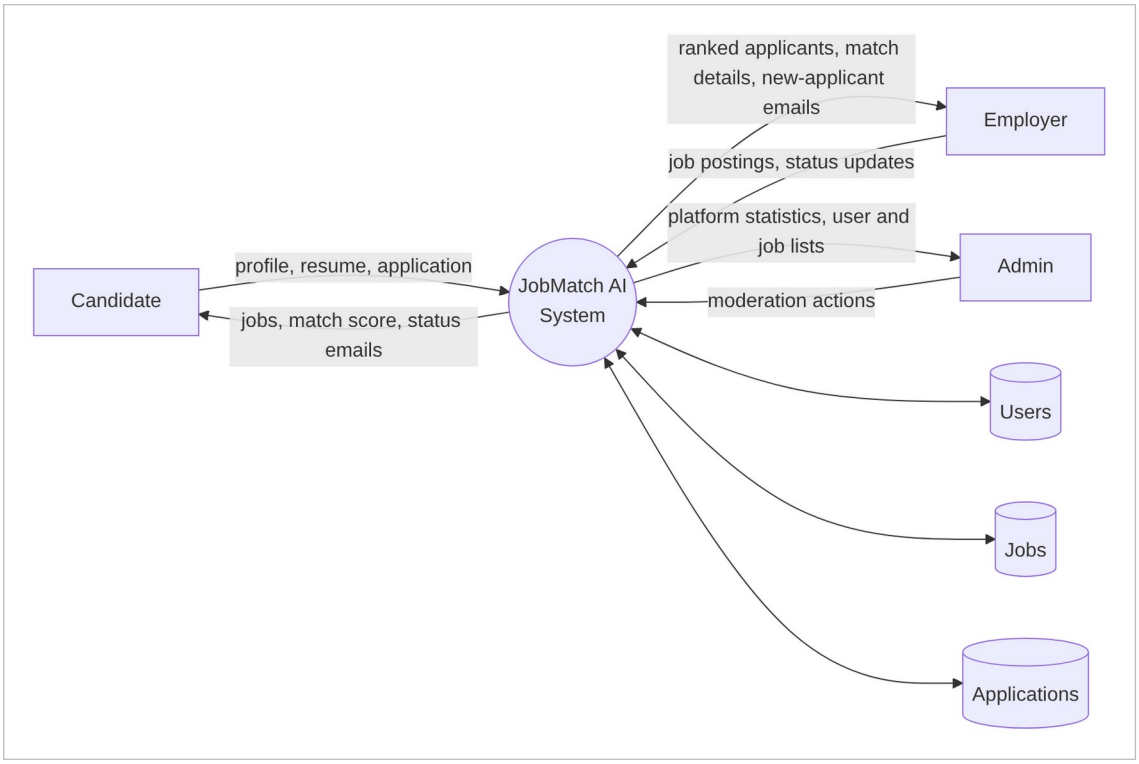


Fig. 5.2 Level-0 data flow diagram

5.3 Use Case Diagram

The use case diagram identifies the main interactions of each actor with the system. Applying with a resume includes AI resume screening and an email notification; updating an applicant's status also includes an email notification.

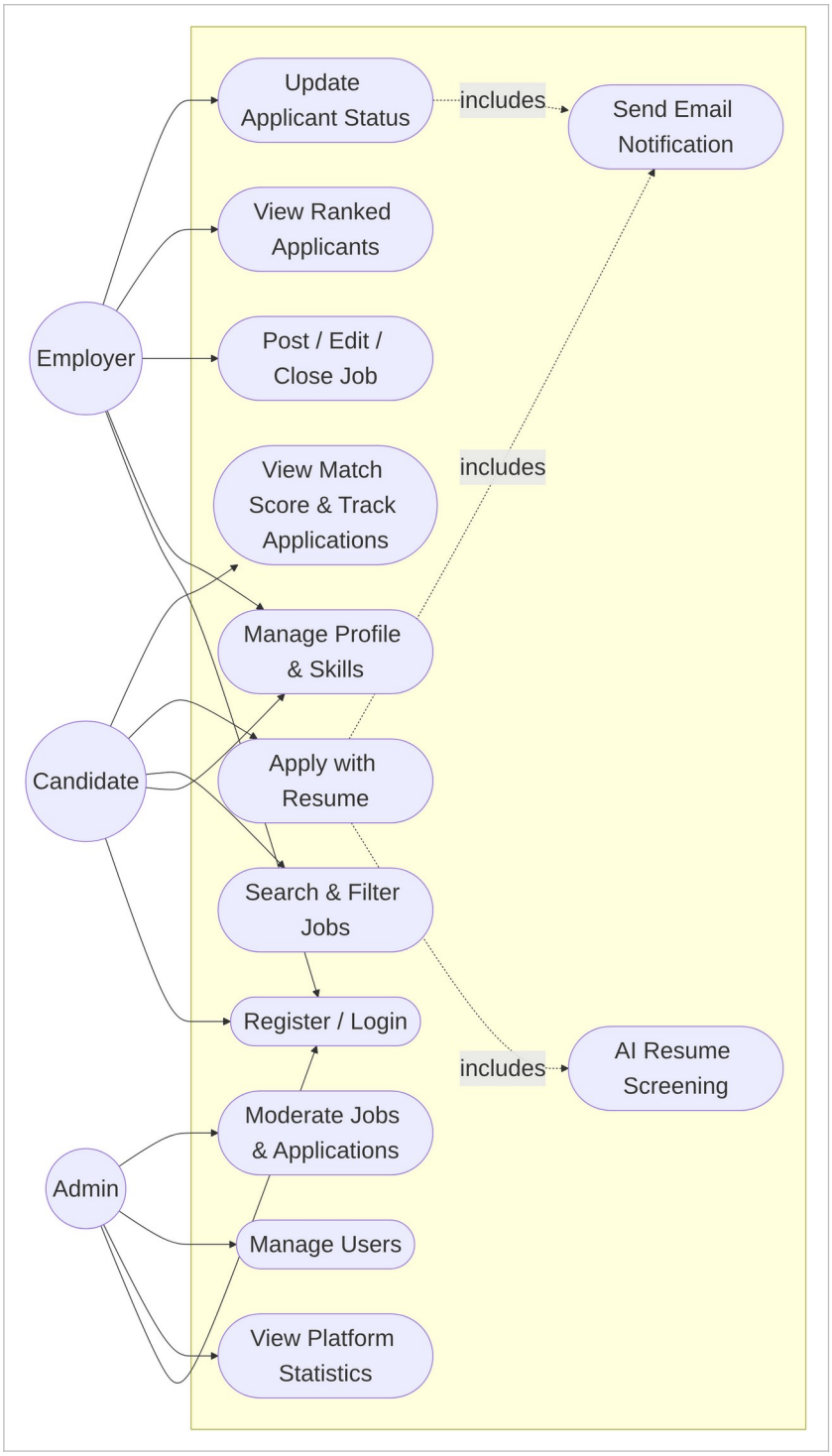


Fig. 5.3 Use case diagram

5.4 Database Design

MongoDB, a document-oriented NoSQL database, was chosen because job postings and AI match details are naturally represented as nested documents and because it integrates seamlessly with Node.js. The database has three collections. A **User**

(employer) posts many **Jobs**; a **User** (candidate) submits many **Applications**; and a **Job** receives many **Applications**.

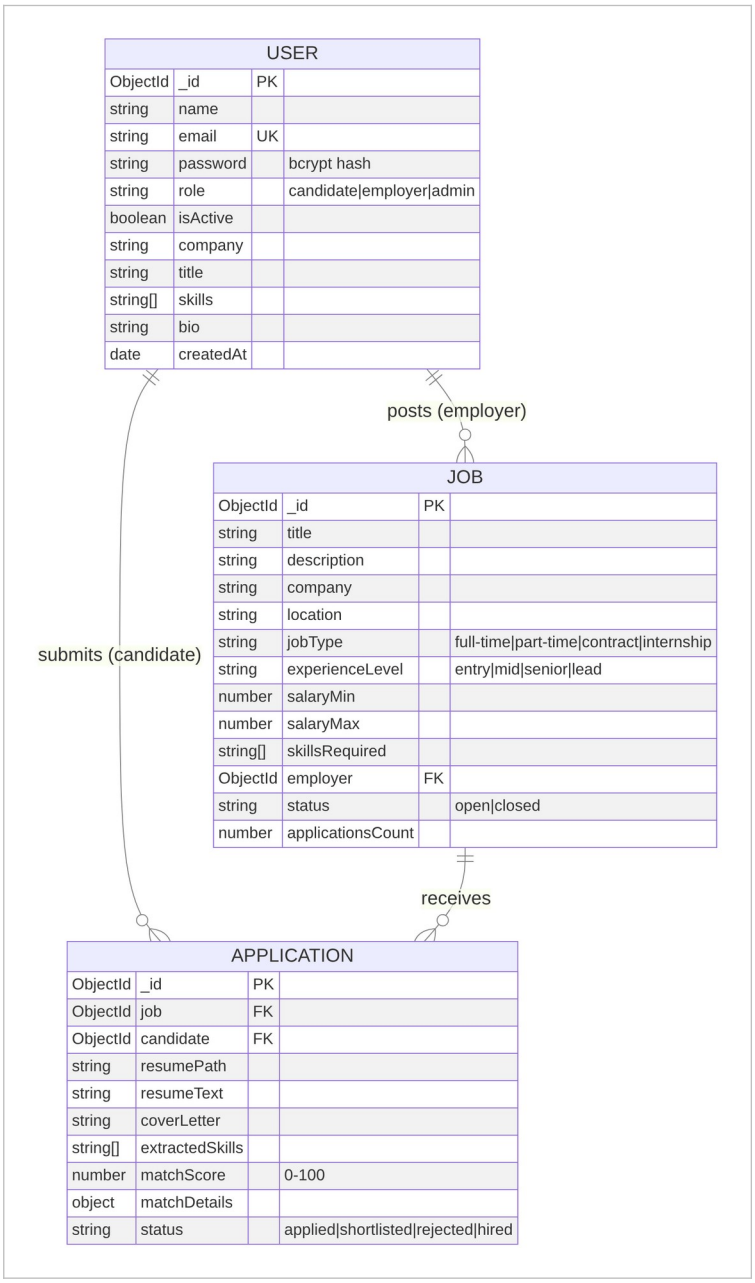


Fig. 5.4 Entity–relationship diagram

Table 5.1 Schema of the users collection

Field	Type	Constraints / Description
_id	ObjectId	Primary key
name	String	Required
email	String	Required, unique, lowercase
password	String	Required, min. 6 characters, bcrypt hash, excluded from queries by default

Field	Type	Constraints / Description
role	String	candidate | employer | admin (default candidate)
isActive	Boolean	Default true; deactivated users cannot log in
company, title, bio	String	Optional profile fields
skills	[String]	Candidate's declared skills
createdAt, updatedAt	Date	Automatic timestamps

Table 5.2 Schema of the jobs collection

Field	Type	Constraints / Description
_id	ObjectId	Primary key
title, description, company	String	Required
location	String	Default "Remote"
jobType	String	full-time | part-time | contract | internship
experienceLevel	String	entry | mid | senior | lead
salaryMin, salaryMax	Number	Optional salary range
skillsRequired	[String]	Lowercase list of required skills
employer	ObjectId	Reference to users (owner)
status	String	open | closed (default open)
applicationsCount	Number	Default 0
Index	—	Text index on title, description and company

Table 5.3 Schema of the applications collection

Field	Type	Constraints / Description
_id	ObjectId	Primary key
job	ObjectId	Reference to jobs, required
candidate	ObjectId	Reference to users, required
resumePath, resumeOriginalName	String	Location and original name of the uploaded file
resumeText	String	Extracted plain text
coverLetter	String	Optional
extractedSkills	[String]	All skills detected/declared for the candidate
matchScore	Number	0–100
matchDetails	Object	skillMatchPercent, textSimilarityPercent, experienceFitPercent, candidateYearsOfExperience, educationLevel, matchedSkills, missingSkills, semanticScore, semanticSummary
status	String	applied | shortlisted | rejected | hired
Index	—	Unique compound index on (job, candidate) to prevent duplicate applications

5.5 Class Diagram

The class diagram shows the three data models and the main service modules of the backend, with their attributes, methods and relationships.

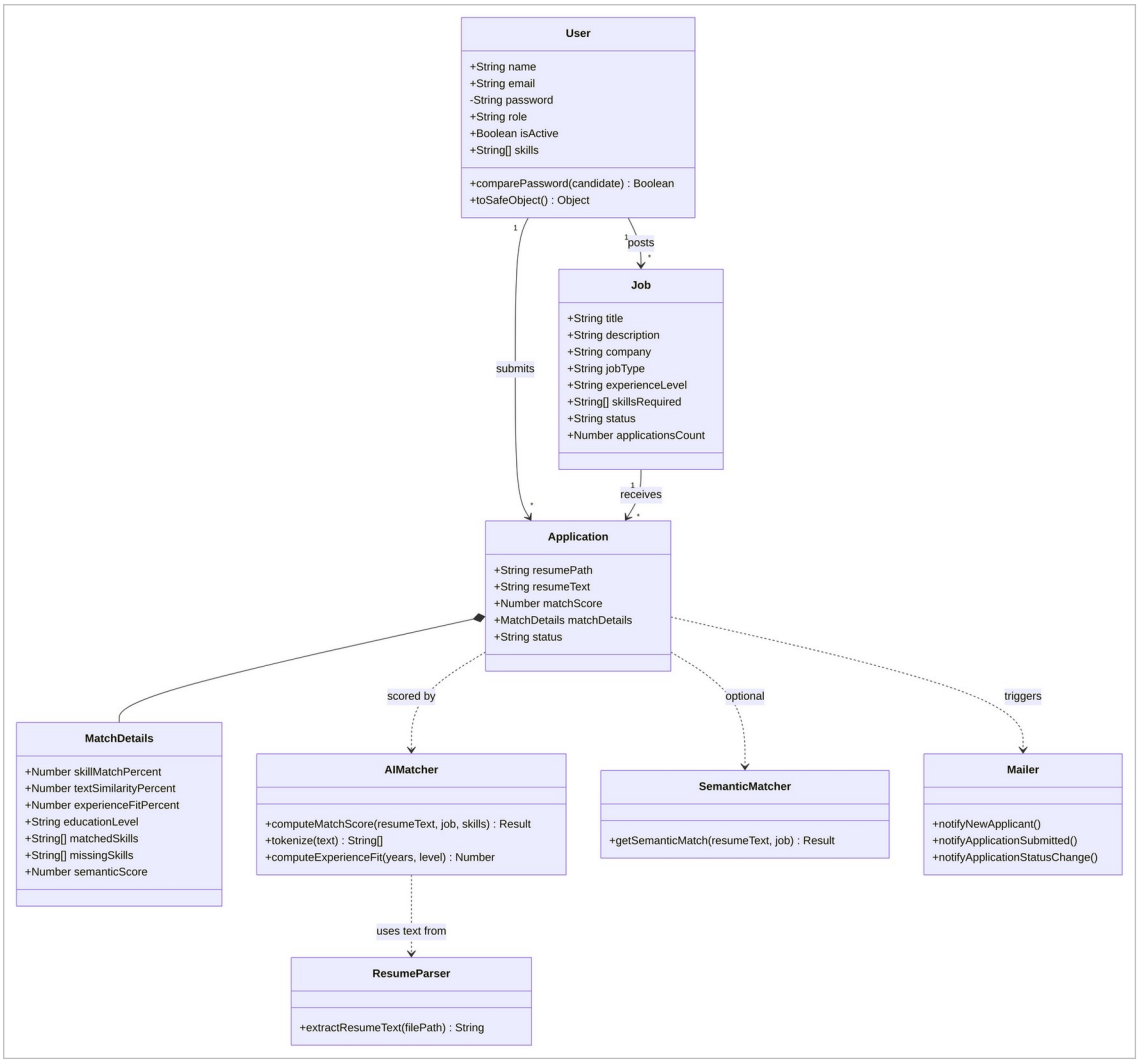


Fig. 5.5 Class diagram

5.6 Sequence Diagram — Applying for a Job

The sequence diagram illustrates the most important workflow: a candidate submitting an application, which triggers resume parsing, AI scoring, storage and notifications.

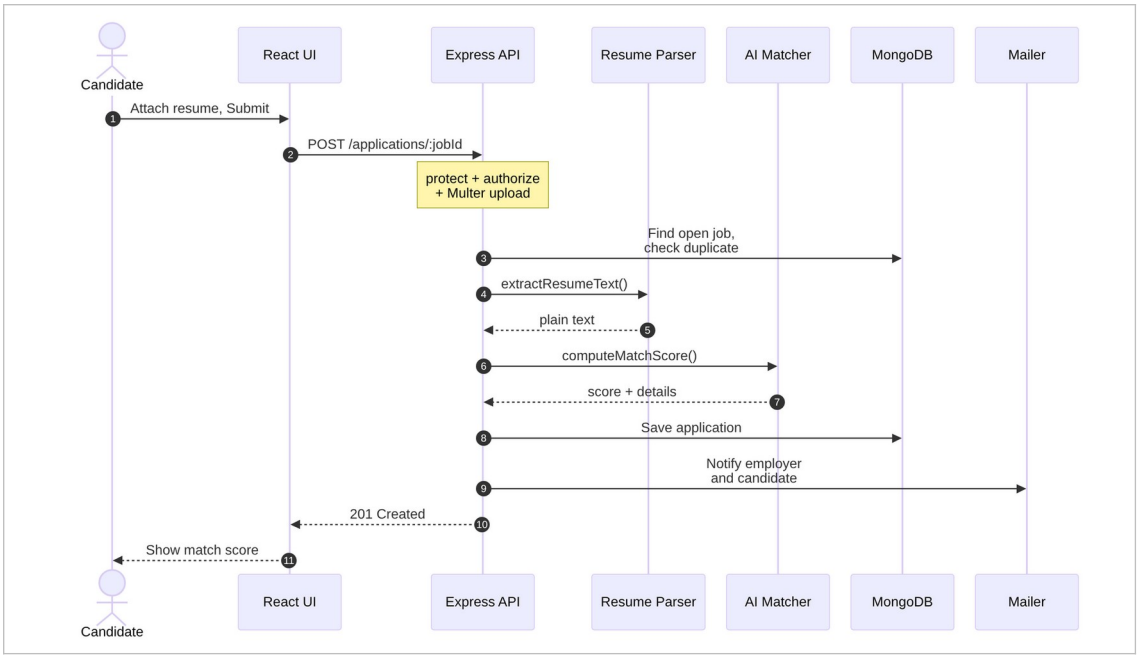


Fig. 5.6 Sequence diagram for job application and AI screening

5.7 Activity Diagram — AI Screening Process

The activity diagram details the decision logic inside the screening engine, including the redistribution of weights when experience is unknown and the optional LLM blend.

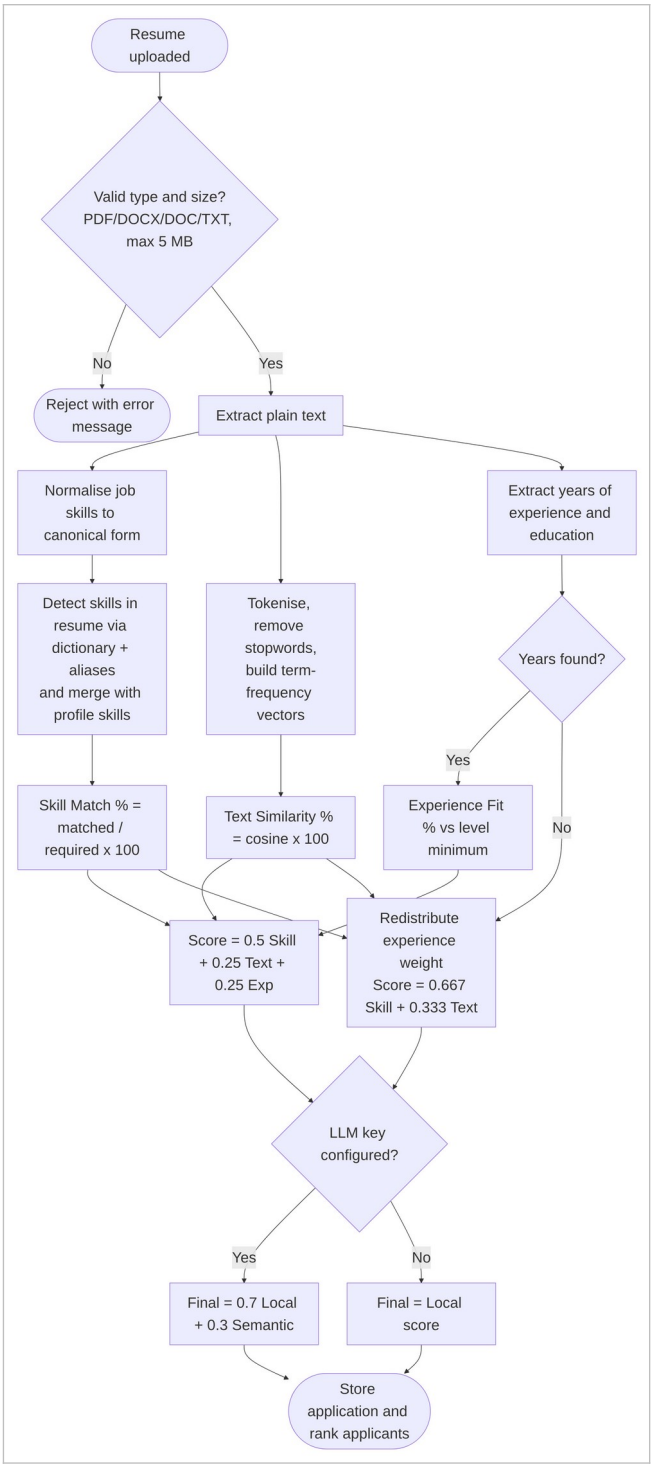


Fig. 5.7 Activity diagram of the AI resume screening process

5.8 REST API Design

Table 5.4 REST API endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register as candidate or employer

Method	Endpoint	Access	Description
POST	/api/auth/login	Public	Log in and receive JWT
GET	/api/auth/me	Authenticated	Current user
PUT	/api/users/me	Authenticated	Update own profile
GET	/api/jobs	Public	Search / list open jobs
GET	/api/jobs/:id	Public	Job details
GET	/api/jobs/employer/mine	Employer	Employer's own postings
POST	/api/jobs	Employer	Create job
PUT / DELETE	/api/jobs/:id	Employer (owner)	Update / delete job
POST	/api/applications/:jobId	Candidate	Apply with resume; runs AI screening
GET	/api/applications/mine	Candidate	Own applications
GET	/api/applications/job/:jobId	Employer (owner)	Ranked applicants
GET	/api/applications/:id	Owner / Admin	Application details
PUT	/api/applications/:id/status	Employer (owner)	Update status; notifies candidate
GET	/api/admin/stats	Admin	Platform statistics
GET / PUT / DELETE	/api/admin/users[/:id]	Admin	List, update, delete users
GET / PUT / DELETE	/api/admin/jobs[/:id]	Admin	List, close/reopen, delete jobs
GET	/api/admin/applications	Admin	All applications

5.9 User Interface Design

The user interface was designed to be clean, consistent and responsive. It uses a single navigation bar whose links change according to the logged-in user's role, card-based layouts for jobs and applicants, and colour-coded badges that communicate the match score at a glance. The following client-side routes are defined:

Table 5.5 Frontend routes and access

Route	Page	Access
/	Home (landing page)	Public
/login, /register	Login, Registration	Public
/jobs, /jobs/:id	Browse Jobs, Job Details and Apply	Public (apply: candidate)
/profile	My Profile	Any logged-in user
/candidate/applications	My Applications	Candidate

Route	Page	Access
/employer/post-job	Post a Job	Employer
/employer/jobs	My Job Postings	Employer
/employer/jobs/:id/edit	Edit Job	Employer
/employer/jobs/:id/applicants	Ranked Applicants	Employer
/admin	Admin Dashboard	Admin

5.10 Security Design

- **Authentication:** stateless JWT tokens signed with a server secret; tokens are verified on every protected request.
- **Password storage:** bcrypt hashing with per-password salt; the password field is never returned by the API.
- **Authorisation:** role checks (candidate, employer, admin) plus ownership checks (an employer can see applicants only for their own jobs).
- **Account control:** deactivated accounts are rejected even if they hold a valid token.
- **File upload safety:** whitelist of file extensions, 5 MB size limit and sanitised, randomised file names.
- **Privacy:** uploaded resumes are not exposed as static files; only extracted text and parsed skills are available through authenticated endpoints.
- **CORS:** API access restricted to the configured client origin.

CHAPTER VI
SAMPLE CODE

This chapter presents selected code excerpts that implement the major functionalities of the system. The complete source code is organised into a `backend` folder (Node.js/Express) and a `frontend` folder (React), together with the analysis notebook.

6.1 AI Matching Engine — Text Processing and Cosine Similarity

The following functions from `backend/utils/aiMatcher.js` tokenise text, build term-frequency maps and compute the cosine similarity between a resume and a job description.

```
function tokenize(text) {
  if (!text) return [];
  return text
    .toLowerCase()
    .replace(/[^a-z0-9+.#\s]/g, ' ')
    .split(/\s+/)
    .filter((tok) => tok.length > 1 && !STOPWORDS.has(tok));
}

function termFrequency(tokens) {
  const freq = new Map();
  for (const tok of tokens) {
    freq.set(tok, (freq.get(tok) || 0) + 1);
  }
  return freq;
}

function cosineSimilarity(freqA, freqB) {
  const vocab = new Set([...freqA.keys(), ...freqB.keys()]);
  let dot = 0;
  let normA = 0;
  let normB = 0;
  for (const term of vocab) {
    const a = freqA.get(term) || 0;
    const b = freqB.get(term) || 0;
    dot += a * b;
    normA += a * a;
    normB += b * b;
  }
  if (normA === 0 || normB === 0) return 0;
  return dot / (Math.sqrt(normA) * Math.sqrt(normB));
}
```

6.2 AI Matching Engine — Score Computation

The `computeMatchScore` function combines the three signals. It canonicalises the job's skills, merges detected and declared candidate skills, computes skill match, text

similarity and experience fit, and redistributes the experience weight when experience is unknown.

```
const SKILL_WEIGHT = 0.5;
const TEXT_WEIGHT = 0.25;
const EXPERIENCE_WEIGHT = 0.25;

// Minimum years of experience typically expected per level, used only to
// gauge experience fit — not a hard requirement.
const EXPERIENCE_LEVEL_MIN_YEARS = {
  entry: 0,
  mid: 2,
  senior: 5,
  lead: 8,
};

function computeExperienceFit(candidateYears, jobExperienceLevel) {
  if (candidateYears === null || candidateYears === undefined) return null;
  const requiredMin = EXPERIENCE_LEVEL_MIN_YEARS[jobExperienceLevel] ?? 0;
  if (requiredMin === 0) return 100;
  if (candidateYears >= requiredMin) return 100;
  return Math.round((candidateYears / requiredMin) * 100);
}

function computeMatchScore(resumeText, job, candidateDeclaredSkills = []) {
  const jobSkills = Array.from(
    new Set((job.skillsRequired || []).map((s) =>
toCanonicalSkill(s)).filter(Boolean))
  );

  const resumeDetectedSkills = extractSkills(resumeText);
  const candidateSkillSet = new Set(
    [...resumeDetectedSkills, ...(candidateDeclaredSkills || [])].map((s) =>
toCanonicalSkill(s))
  );

  const matchedSkills = jobSkills.filter((skill) =>
candidateSkillSet.has(skill));
  const missingSkills = jobSkills.filter((skill) => !
candidateSkillSet.has(skill));
  const skillMatchPercent =
    jobSkills.length === 0 ? 100 : Math.round((matchedSkills.length /
jobSkills.length) * 100);

  const jobText = `${job.title || ''} ${job.description || ''} $
{jobSkills.join(' ')}`;
  const resumeFreq = termFrequency(tokenize(resumeText));
  const jobFreq = termFrequency(tokenize(jobText));
  const similarity = cosineSimilarity(resumeFreq, jobFreq);
  const textSimilarityPercent = Math.round(similarity * 100);

  const candidateYears = extractYearsOfExperience(resumeText);
  const educationLevel = extractEducationLevel(resumeText);
  const experienceFitPercent = computeExperienceFit(candidateYears,
job.experienceLevel);

  // Redistribute the experience weight onto skills/text when experience
  // couldn't be determined, so the score stays on a comparable 0-100 scale.
  let score;
  if (experienceFitPercent === null) {
    const redistributed = SKILL_WEIGHT + EXPERIENCE_WEIGHT * (SKILL_WEIGHT /
(SKILL_WEIGHT + TEXT_WEIGHT));
```

```
    const redistributedText = TEXT_WEIGHT + EXPERIENCE_WEIGHT * (TEXT_WEIGHT /
(SKILL_WEIGHT + TEXT_WEIGHT));
    score = Math.round(redistributed * skillMatchPercent + redistributedText *
textSimilarityPercent);
  } else {
    score = Math.round(
      SKILL_WEIGHT * skillMatchPercent +
        TEXT_WEIGHT * textSimilarityPercent +
        EXPERIENCE_WEIGHT * experienceFitPercent
    );
  }

  return {
    score: Math.min(100, Math.max(0, score)),
    skillMatchPercent,
    textSimilarityPercent,
    experienceFitPercent,
    candidateYearsOfExperience: candidateYears,
    educationLevel,
    matchedSkills,
    missingSkills,
    extractedSkills: Array.from(candidateSkillSet),
  };
}
```

6.3 Skill Alias Resolution

An alias table in `backend/utils/skillAliases.js` maps common abbreviations and synonyms to a canonical skill name, so that equivalent terms are matched correctly.

```
const SKILL_ALIASES = {
  javascript: ['js', 'es6', 'ecmascript'],
  typescript: ['ts'],
  'node.js': ['node', 'nodejs'],
  'react.js': ['react', 'reactjs'],
  'vue.js': ['vue', 'vuejs'],
  'next.js': ['nextjs', 'next'],
  'express.js': ['express', 'expressjs'],
  mongodb: ['mongo'],
  postgresql: ['postgres', 'psql'],
  kubernetes: ['k8s'],
  'machine learning': ['ml'],
  'deep learning': ['dl'],
  'natural language processing': ['nlp'],
  'artificial intelligence': ['ai'],
  'continuous integration': ['ci/cd', 'ci', 'cd'],
  'ui/ux': ['ux', 'ui', 'ux/ui'],
  'google cloud': ['gcp'],
  'amazon web services': ['aws'],
  golang: ['go'],
  'c#': ['csharp', 'c-sharp'],
  'c++': ['cpp'],
  'ruby on rails': ['rails'],
  rest: ['restful', 'rest api', 'restful api'],
  sql: ['structured query language'],
};

const ALIAS_TO_CANONICAL = Object.entries(SKILL_ALIASES).reduce((acc,
[canonical, aliases]) => {
  for (const alias of aliases) {
    acc[alias.toLowerCase()] = canonical;
```

```
  }
  return acc;
}, {});
```

6.4 Experience and Education Extraction

Regular-expression patterns in `backend/utils/experienceParser.js` detect the number of years of experience mentioned in the resume.

```
const YEARS_PATTERNS = [
  /(\d{1,2})\s*\+?\s*(?:years?|yrs?)\s*(?:of)?\s*(?:professional\s*)?
experience/gi,
  /experience\s*(?:of)?\s*(\d{1,2})\s*\+?\s*(?:years?|yrs?)/gi,
  /(\d{1,2})\s*\+?\s*(?:years?|yrs?)\s*(?:in|as|working)/gi,
];

function extractYearsOfExperience(text) {
  if (!text) return null;
  let max = null;
  for (const pattern of YEARS_PATTERNS) {
    for (const match of text.matchAll(pattern)) {
      const years = parseInt(match[1], 10);
      if (!Number.isNaN(years) && years >= 0 && years <= 50) {
        max = max === null ? years : Math.max(max, years);
      }
    }
  }
  return max;
}
```

6.5 Resume Text Extraction

The parser in `backend/utils/resumeParser.js` selects an extraction method according to the file type.

```
async function extractResumeText(filePath) {
  const ext = path.extname(filePath).toLowerCase();
  const buffer = fs.readFileSync(filePath);

  if (ext === '.pdf') {
    const data = await pdfParse(buffer);
    return data.text || '';
  }

  if (ext === '.docx') {
    const result = await mammoth.extractRawText({ buffer });
    return result.value || '';
  }

  if (ext === '.txt' || ext === '.doc') {
    return buffer.toString('utf-8');
  }

  throw new Error(`Unsupported file type for parsing: ${ext}`);
}
```

6.6 Secure Resume Upload Configuration

Multer is configured in `backend/middleware/upload.js` to store files with sanitised unique names, allow only resume formats and limit the file size to 5 MB.

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, uploadDir),
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    const safeBase = path
      .basename(file.originalname, ext)
      .replace(/[^a-zA-Z0-9_-]/g, '_')
      .slice(0, 40);
    const unique = `${Date.now()}-${Math.round(Math.random() * 1e9)}`;
    cb(null, `${safeBase}-${unique}${ext}`);
  },
});

const allowedExt = new Set(['.pdf', '.docx', '.doc', '.txt']);

const fileFilter = (req, file, cb) => {
  const ext = path.extname(file.originalname).toLowerCase();
  if (allowedExt.has(ext)) {
    cb(null, true);
  } else {
    cb(new Error('Unsupported resume format. Please upload a PDF, DOC, DOCX, or
TXT file.'));
  }
};

const upload = multer({
  storage,
  fileFilter,
  limits: { fileSize: 5 * 1024 * 1024 },
});
```

6.7 Authentication and Role-Based Authorisation Middleware

```
const protect = async (req, res, next) => {
  try {
    const header = req.headers.authorization;
    if (!header || !header.startsWith('Bearer ')) {
      return res.status(401).json({ message: 'Not authorized, no token provided'
});
    }
    const token = header.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id);
    if (!user) {
      return res.status(401).json({ message: 'User no longer exists' });
    }
    if (!user.isActive) {
      return res.status(403).json({ message: 'This account has been deactivated'
});
    }
    req.user = user;
    next();
  } catch (err) {
    return res.status(401).json({ message: 'Not authorized, invalid or expired
token' });
  }
};
```

```
const authorize = (...roles) => (req, res, next) => {
  if (!req.user || !roles.includes(req.user.role)) {
    return res.status(403).json({ message: 'Forbidden: insufficient role
permissions' });
  }
  next();
};
```

6.8 Application Controller — Apply to Job with AI Screening

This controller in `backend/controllers/applicationController.js` orchestrates the complete application workflow: validation, duplicate check, resume parsing, local AI scoring, the optional LLM blend, storage and notifications. (Some lines are abridged for brevity.)

```
const LOCAL_WEIGHT_WITH_SEMANTIC = 0.7;
const SEMANTIC_WEIGHT = 0.3;

const applyToJob = async (req, res, next) => {
  try {
    const job = await Job.findById(req.params.jobId);
    if (!job) {
      if (req.file) fs.unlink(req.file.path, () => {});
      return res.status(404).json({ message: 'Job not found' });
    }
    if (job.status !== 'open') {
      if (req.file) fs.unlink(req.file.path, () => {});
      return res.status(400).json({ message: 'This job is no longer accepting
applications' });
    }
    if (!req.file) {
      return res.status(400).json({ message: 'A resume file is required' });
    }

    const existing = await Application.findOne({ job: job._id, candidate:
req.user._id });
    if (existing) {
      fs.unlink(req.file.path, () => {});
      return res.status(409).json({ message: 'You have already applied to this
job' });
    }

    let resumeText = '';
    try {
      resumeText = await extractResumeText(req.file.path);
    } catch (parseErr) {
      fs.unlink(req.file.path, () => {});
      return res.status(422).json({
        message: `Could not parse resume: ${parseErr.message}`,
      });
    }

    const matchResult = computeMatchScore(resumeText, job, req.user.skills);

    const semanticMatch = await getSemanticMatch(resumeText, job);
    const finalScore = semanticMatch
      ? Math.round(
```

```
          LOCAL_WEIGHT_WITH_SEMANTIC * matchResult.score + SEMANTIC_WEIGHT *
semanticMatch.score
        )
      : matchResult.score;
    // ... Application.create({...}), applicationsCount++, email
notifications ...
```

6.9 Ranked Applicants Query

```
const getApplicantsForJob = async (req, res, next) => {
  try {
    const job = await Job.findById(req.params.jobId);
    if (!job) return res.status(404).json({ message: 'Job not found' });
    if (job.employer.toString() !== req.user._id.toString()) {
      return res.status(403).json({ message: 'You do not own this job
posting' });
    }

    const applications = await Application.find({ job: job._id })
      .populate('candidate', 'name email title skills resumePath')
      .sort({ matchScore: -1, createdAt: -1 });

    res.json({ job: { _id: job._id, title: job.title }, applications });
  } catch (err) {
    next(err);
  }
};
```

6.10 Application Data Model

```
const applicationSchema = new mongoose.Schema(
  {
    job: { type: mongoose.Schema.Types.ObjectId, ref: 'Job', required: true },
    candidate: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
    resumePath: { type: String, required: true },
    resumeOriginalName: { type: String },
    resumeText: { type: String },
    coverLetter: { type: String, trim: true },
    extractedSkills: [{ type: String, trim: true, lowercase: true }],
    matchScore: { type: Number, default: 0, min: 0, max: 100 },
    matchDetails: {
      skillMatchPercent: { type: Number, default: 0 },
      textSimilarityPercent: { type: Number, default: 0 },
      experienceFitPercent: { type: Number, default: null },
      candidateYearsOfExperience: { type: Number, default: null },
      educationLevel: { type: String, default: null },
      matchedSkills: [{ type: String }],
      missingSkills: [{ type: String }],
      semanticScore: { type: Number, default: null },
      semanticSummary: { type: String, default: null },
    },
    status: {
      type: String,
      enum: ['applied', 'shortlisted', 'rejected', 'hired'],
      default: 'applied',
    },
  },
  { timestamps: true }
);

applicationSchema.index({ job: 1, candidate: 1 }, { unique: true });
```

6.11 Frontend — Match Score Badge Component

The React component below colour-codes the match score shown to candidates and employers.

```
export default function MatchScoreBadge({ score }) {
  let colorClasses = 'bg-slate-100 text-slate-700';
  if (score >= 75) colorClasses = 'bg-emerald-100 text-emerald-700';
  else if (score >= 50) colorClasses = 'bg-amber-100 text-amber-700';
  else if (score >= 0) colorClasses = 'bg-rose-100 text-rose-700';

  return (
    <span className={`inline-flex items-center gap-1 rounded-full px-2.5 py-1
text-xs font-semibold ${colorClasses}`}>
      {score}% AI Match
    </span>
  );
}
```

6.12 Frontend — Submitting an Application

```
  const handleApply = async (e) => {
    e.preventDefault();
    if (!resumeFile) {
      setApplyStatus({ type: 'error', message: 'Please attach a resume (PDF,
DOCX, or TXT).' });
      return;
    }
    setSubmitting(true);
    setApplyStatus(null);
    try {
      const formData = new FormData();
      formData.append('resume', resumeFile);
      if (coverLetter) formData.append('coverLetter', coverLetter);
      const { data } = await api.post(`/applications/${id}`, formData, {
        headers: { 'Content-Type': 'multipart/form-data' },
      });
      setApplyStatus({
        type: 'success',
        message: `Application submitted! Your AI match score: $
{data.application.matchScore}%`,
      });
    } catch (err) {
      setApplyStatus({ type: 'error', message: err.response?.data?.message ||
'Failed to apply' });
    } finally {
      setSubmitting(false);
    }
  };
```

6.13 Recruitment Analytics — Pre-processing and Random Forest Pipeline (Python)

```
numeric_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]
)
```

```
categorical_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(
            handle_unknown="ignore",
            sparse_output=False
        ))
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("numeric", numeric_pipeline, numerical_features),
        ("categorical", categorical_pipeline, categorical_features)
    ]
)

# ============================================================
# 12. RANDOM FOREST MODEL
# ============================================================

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),

        ("classifier", RandomForestClassifier(
            n_estimators=300,
            random_state=42,
            n_jobs=-1,
            class_weight="balanced"
        ))
    ]
)
```

CHAPTER VII
OUTPUT

This chapter presents the output screens of the developed system. The screenshots were captured from the running React application with a set of demonstration data (three job postings and four sample candidates). The match scores shown were computed by the project's actual AI matching engine on the sample resumes.

7.1 Home Page

The landing page introduces the platform and its three key benefits — smart screening, ranked applicants and simplicity for everyone — with calls to action to browse jobs or sign up.

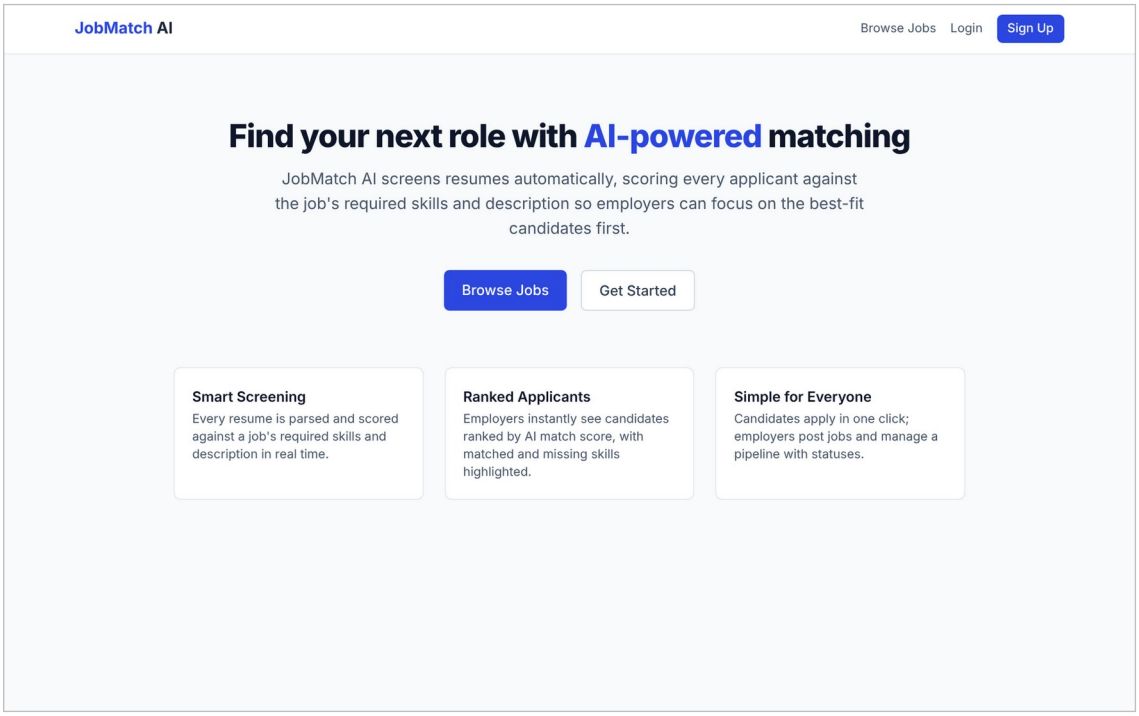


Fig. 7.1 Home page

7.2 Registration and Login

New users register by choosing the candidate or employer role and entering their name, email and password. Registered users log in with email and password; after login they are redirected to the page appropriate for their role.

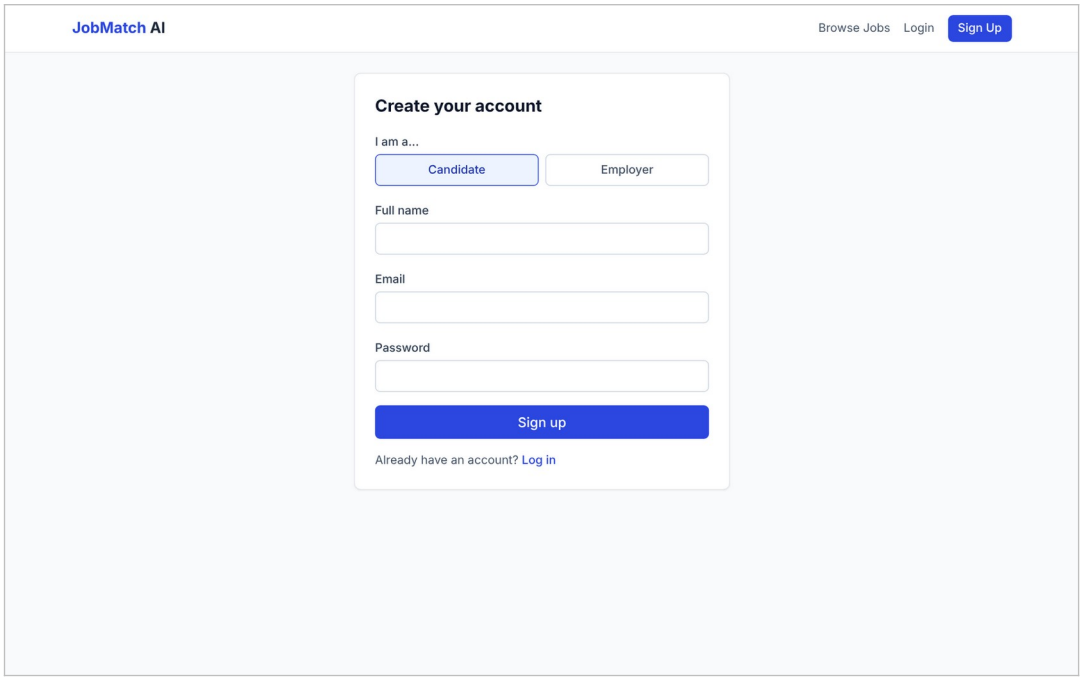


Fig. 7.2 Registration page with role selection

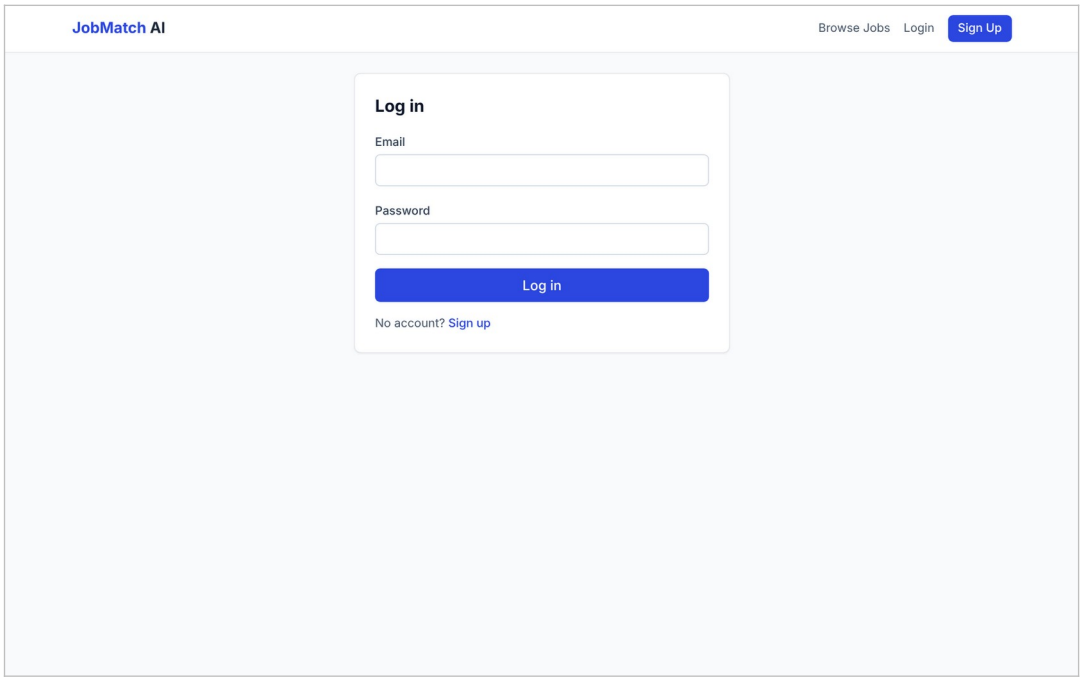


Fig. 7.3 Login page

7.3 Browse and Search Jobs

Candidates can search open jobs by keyword and filter them by location and job type. Each job card shows the title, company, location, job type and required skills.

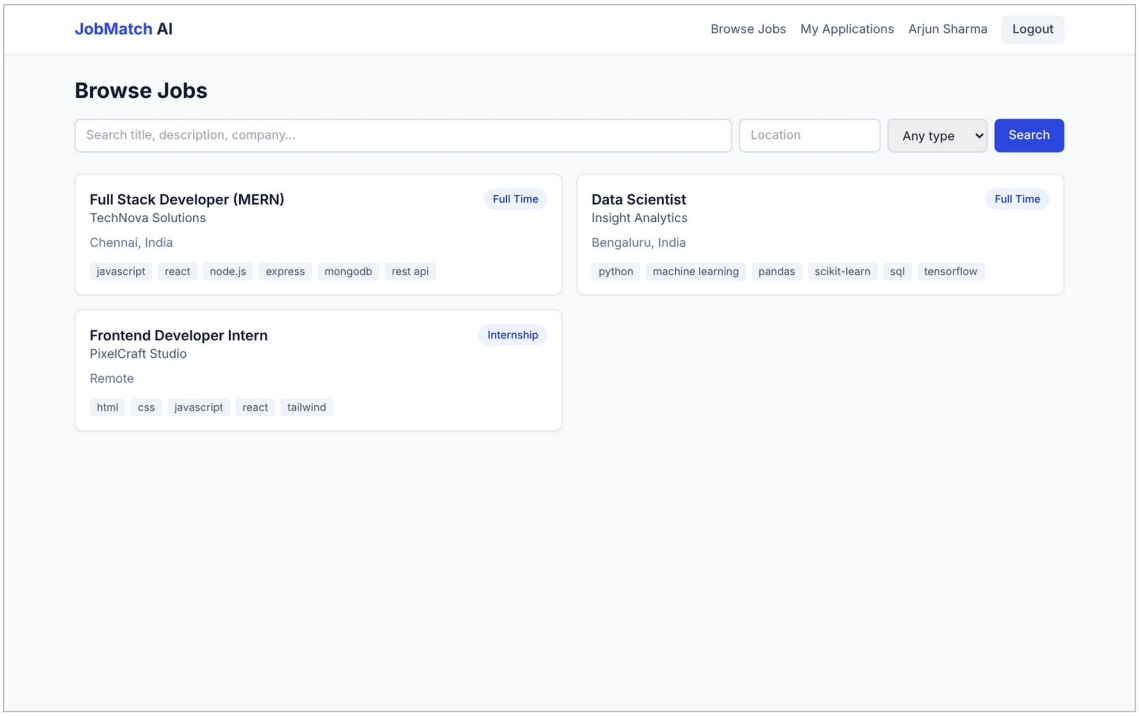


Fig. 7.4 Browse Jobs page with search and filters

7.4 Job Details and Application

The job details page shows the complete description, job type, experience level, salary range and required skills. A candidate attaches a resume, optionally writes a cover letter and submits the application. The AI match score is displayed immediately after submission.

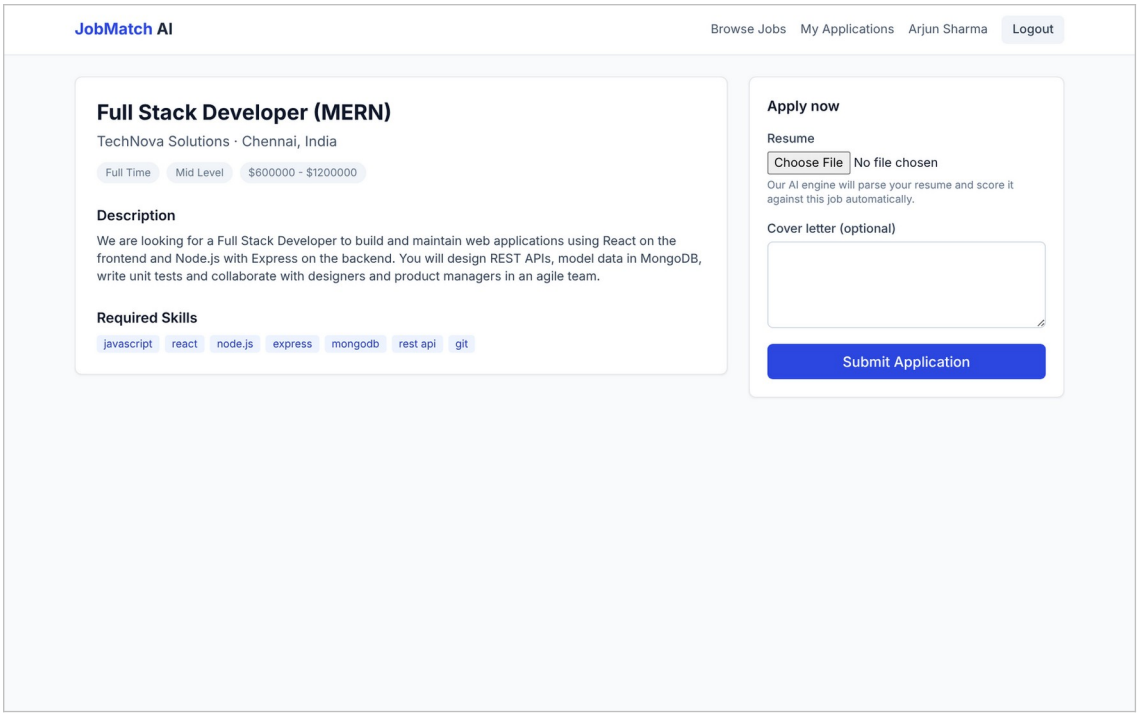


Fig. 7.5 Job details page with the application form

7.5 Candidate — My Applications

Candidates can track all their applications with the AI match score and current status of each.

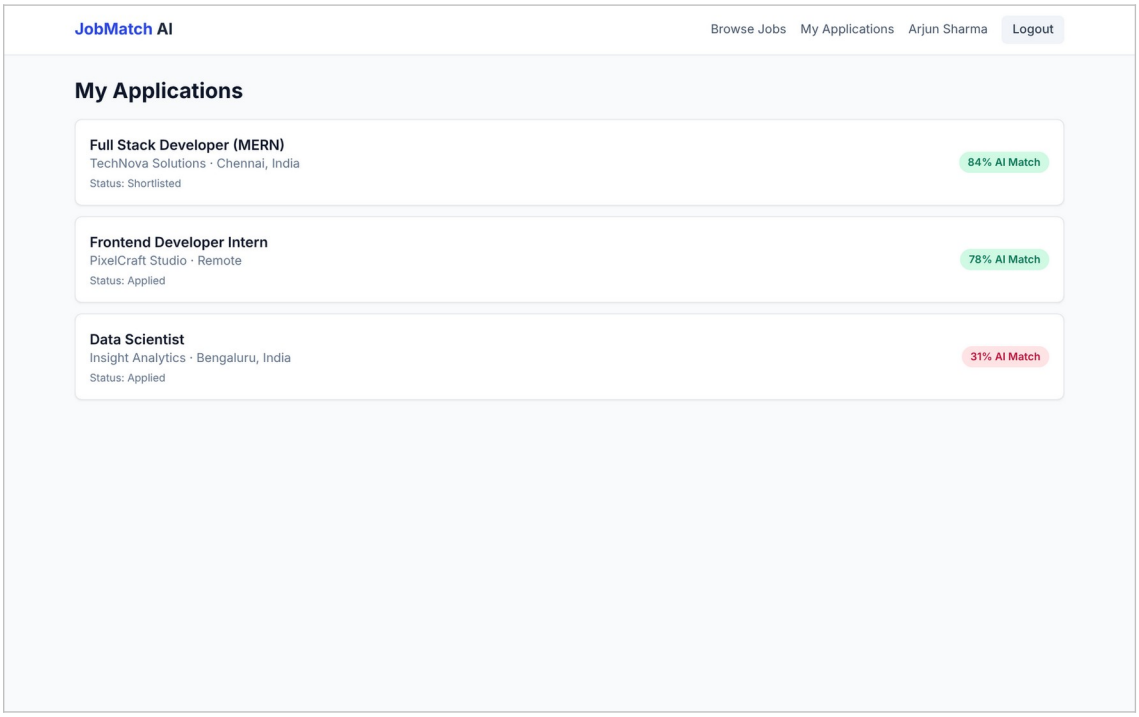


Fig. 7.6 Candidate's application tracking page

7.6 User Profile

Users maintain their name, headline, skills and bio. Skills entered here are combined with the skills detected in the resume during AI screening.

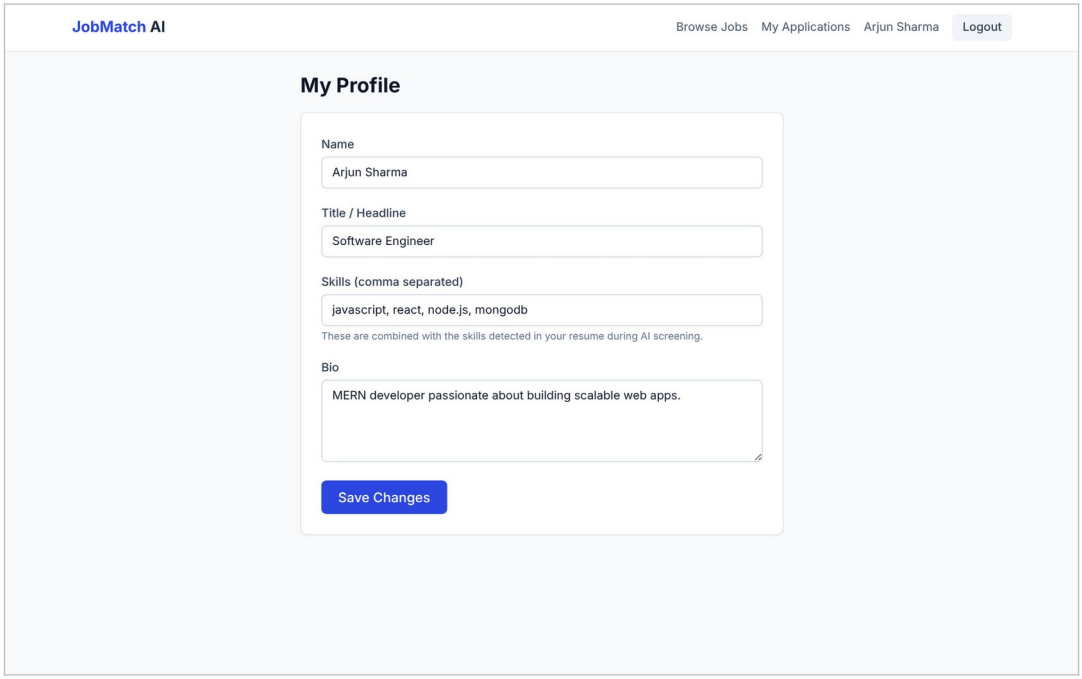


Fig. 7.7 Profile page

7.7 Employer — Post a Job

Employers create a job by entering the title, company, location, job type, experience level, salary range, description and required skills. The required skills are used by the AI engine to score every applicant.

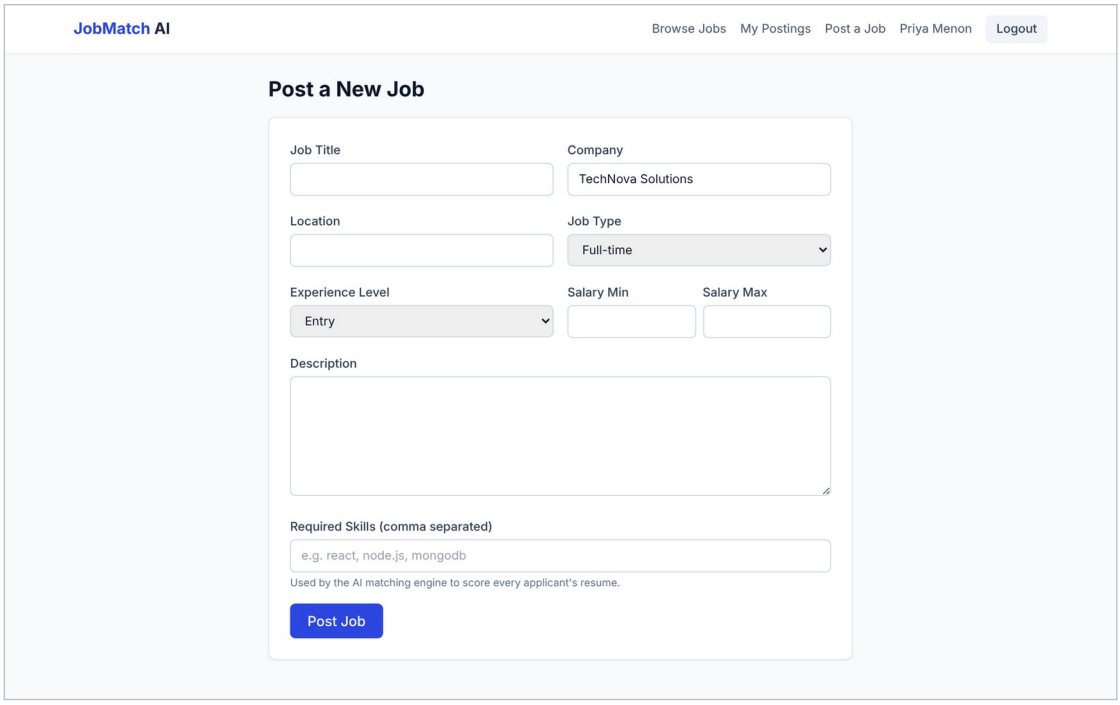


Fig. 7.8 Post a New Job form

7.8 Employer — My Job Postings

Employers see all their postings with the number of applicants and status, and can view applicants, edit, close/reopen or delete each job.

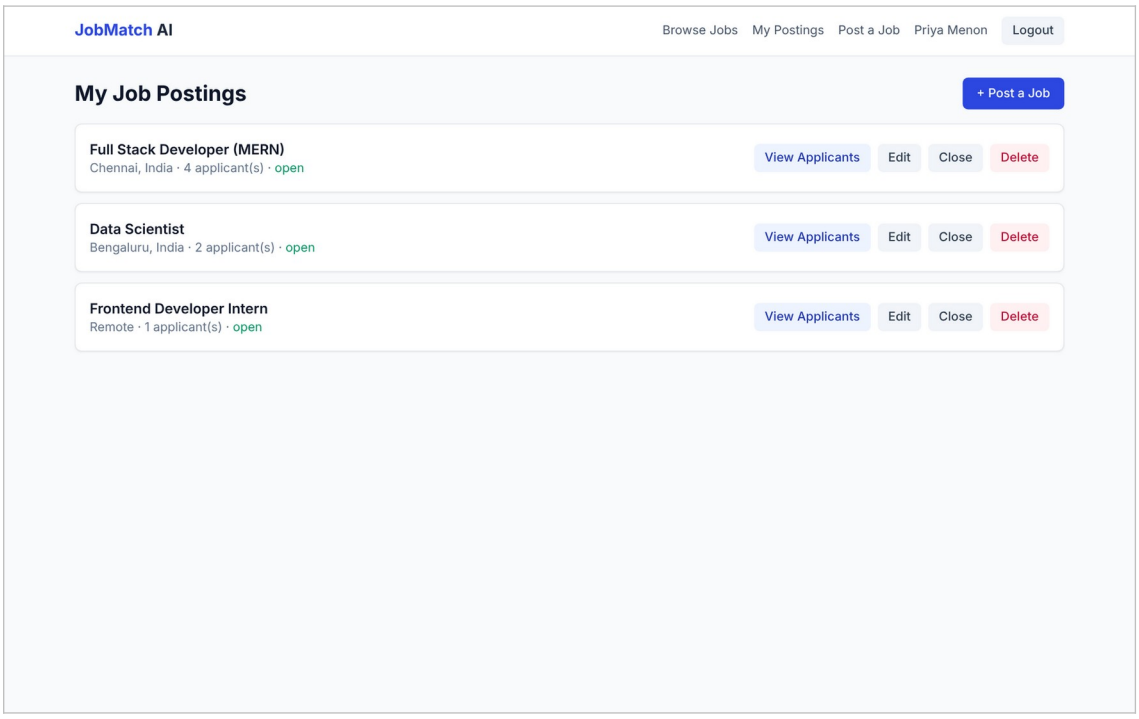


Fig. 7.9 Employer's job postings

7.9 Employer — Ranked Applicants with AI Match Details

This is the key output of the project. Applicants are listed in descending order of AI match score. The details of the top candidate are expanded, showing a 100% skill match with all seven required skills matched, 34% resume/description similarity, 100% experience fit (4 years mentioned against a mid-level role), a detected Bachelor's degree and the cover letter. The employer can change each applicant's status from the drop-down list.

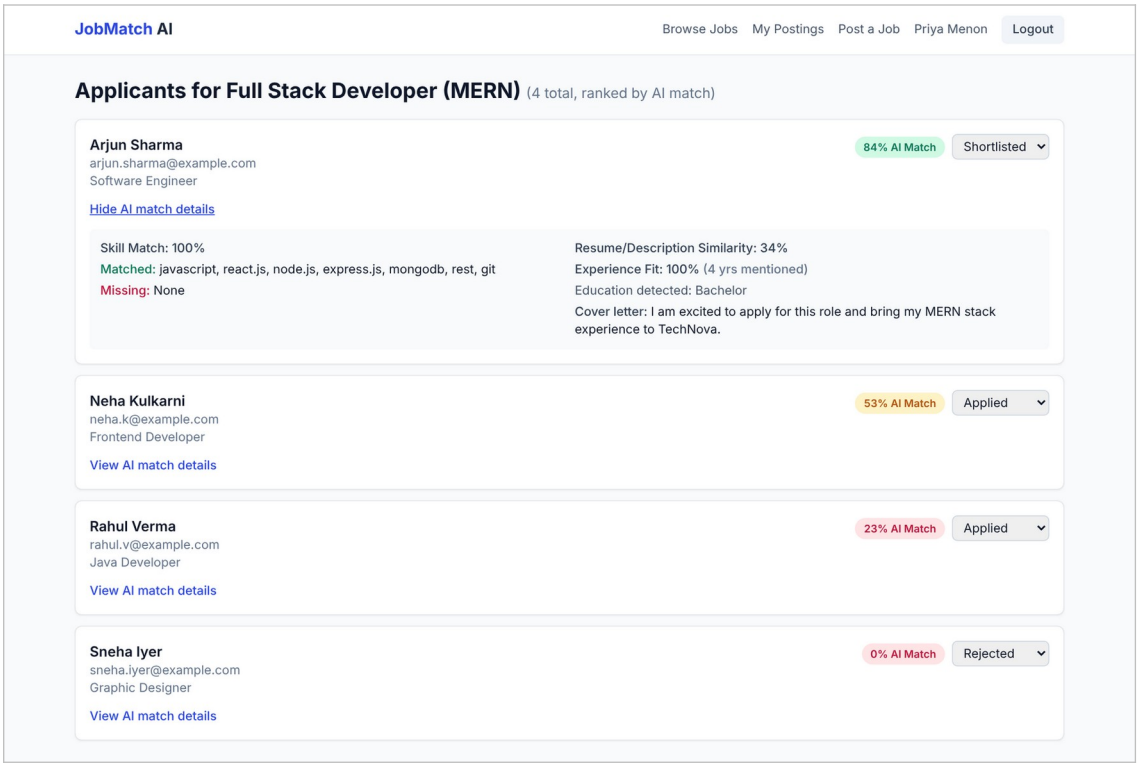


Fig. 7.10 Ranked applicants with AI match breakdown

7.10 Administrator Dashboard

The administrator sees platform-wide statistics and can manage users, jobs and applications. In the Users tab, accounts can be searched, filtered by role, activated, deactivated or deleted; deactivated accounts are clearly labelled.

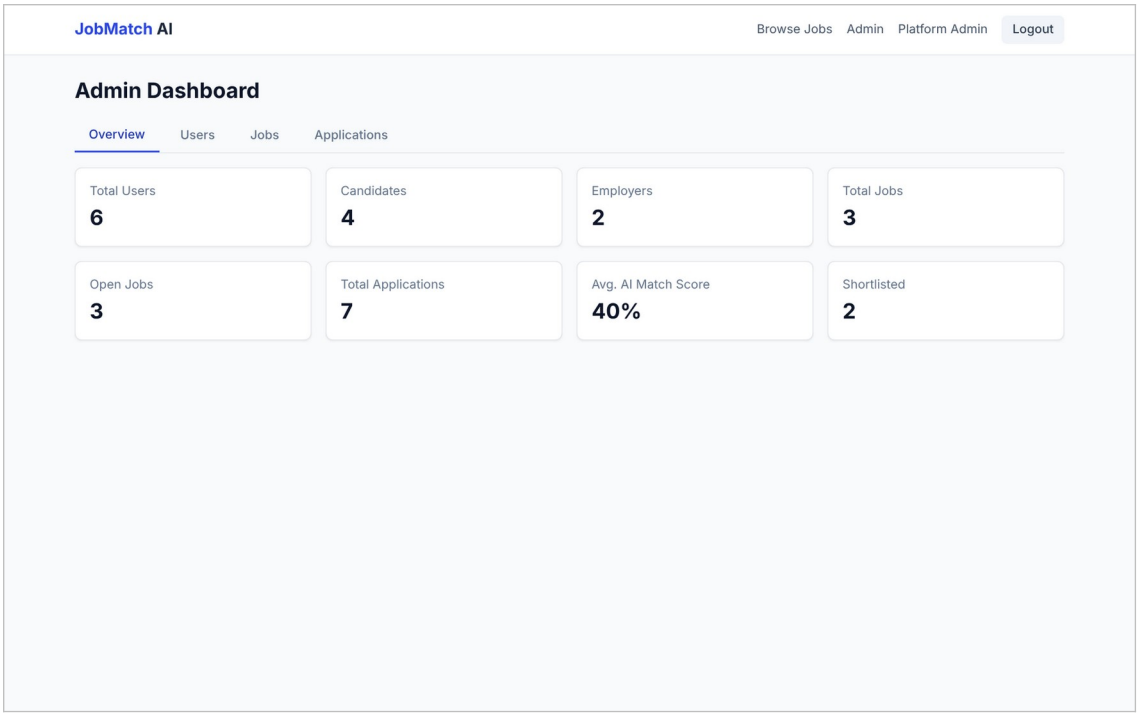


Fig. 7.11 Admin dashboard — overview statistics

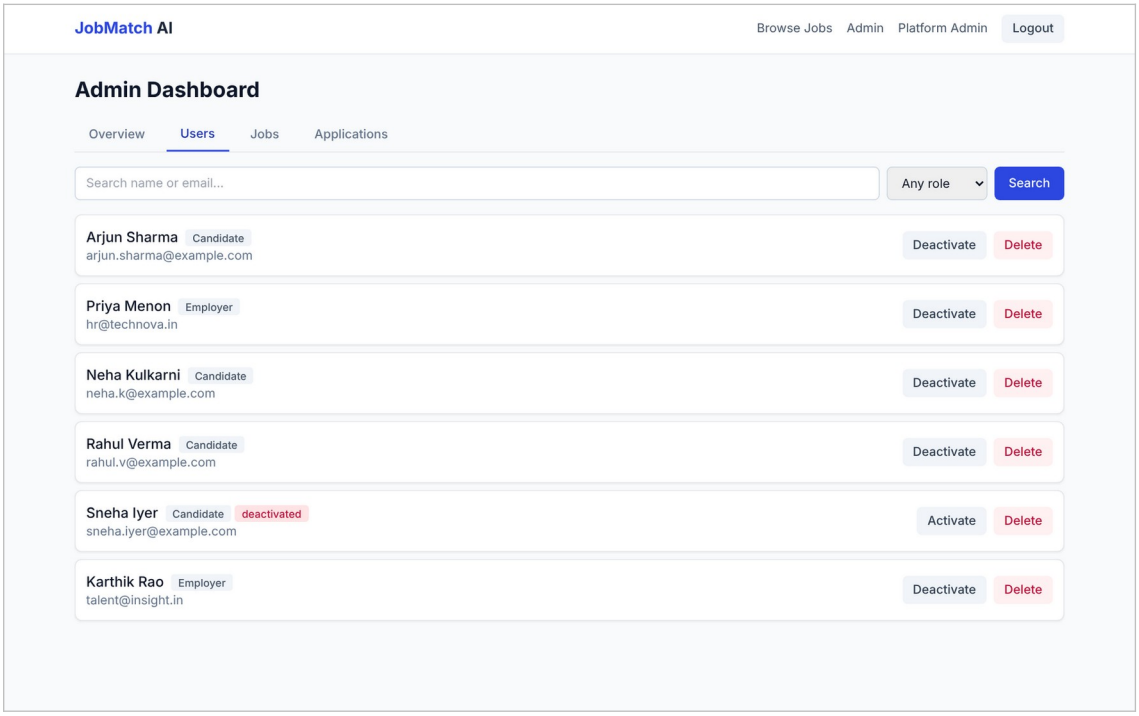


Fig. 7.12 Admin dashboard — user management

7.11 Sample AI Screening Results

Four sample resumes were screened against the job "Full Stack Developer (MERN)" (mid level; required skills: javascript, react, node.js, express, mongodb, rest api, git). The results produced by the matching engine are shown below.

Table 7.1 AI screening results for the sample job

Candidate	Profile Summary	Skill %	Text %	Exp. Fit %	Score
Arjun Sharma	Software engineer, 4 yrs, MERN stack, REST, Git, B.Tech	100	34	100	84
Neha Kulkarni	Frontend developer, 2 yrs, React, JavaScript, HTML/CSS, Tailwind, B.Sc	43	24	100	53
Rahul Verma	Java developer, 1 yr, Spring Boot, MySQL, Git, B.E.	14	13	50	23
Sneha Iyer	Graphic designer, Photoshop, Illustrator (no years stated)	0	0	Unknown	0

Worked example (Arjun Sharma): all 7 required skills were found (Skill = 100%); the TF cosine similarity between the resume and the job text was 0.34 (Text = 34%); 4 years of experience exceeds the 2-year minimum for a mid-level role (Experience = 100%). Hence Score = 0.50 × 100 + 0.25 × 34 + 0.25 × 100 = 50 + 8.5 + 25 = 83.5, rounded to **84%**.

Worked example (Rahul Verma): 1 of 7 skills matched (git), giving 14%; text similarity 13%; 1 year against a 2-year minimum gives 50% experience fit. Score = 0.50 × 14 + 0.25 × 13 + 0.25 × 50 = 7 + 3.25 + 12.5 = 22.75 ≈ **23%**.

The ranking — full-stack developer first, frontend developer second, Java developer third and graphic designer last — matches the judgement an experienced recruiter would make, and the missing-skills list tells the employer exactly why each candidate scored lower (for example, Neha is missing node.js, express, mongodb and git). This confirms that the system meets its objective of producing an accurate, explainable first-pass shortlist.

CHAPTER VIII
TESTING

8.1 Testing Strategy

A multi-level testing strategy was adopted to verify that each part of the system works correctly on its own and in combination with the others:

- **Unit testing** of pure functions and modules with mocked dependencies — the matching engine, skill extraction and aliasing, experience and education parsing, authentication middleware, mailer and semantic layer.
- **Integration testing** of the complete HTTP API against a real, in-memory MongoDB instance (mongodb-memory-server) using Supertest, covering registration, login, authorisation, job posting, applying with a resume, ranking, status updates and administration.
- **Component testing** of React components, context and pages using Vitest and React Testing Library.
- **Manual / user-acceptance testing** of end-to-end flows in the browser for each role.

Table 8.1 Testing tools

Tool	Purpose
Jest 29	Backend test runner and assertion library
Supertest	HTTP assertions against the Express app
mongodb-memory-server	Temporary in-memory MongoDB for integration tests
Vitest 2	Frontend test runner (Vite-native)
React Testing Library, jsdom	Rendering and interacting with React components in a simulated browser

8.2 Unit Test Cases — AI Matching Engine and Parsers

Table 8.2 Unit test cases for the AI matching engine and parsers

ID	Test Case	Expected Result	Status
UT01	Score a strong, relevant resume against a job	High score with required skills matched	Pass
UT02	Score an irrelevant resume	Score at or near zero	Pass
UT03	Resume says "js", job requires "javascript"	Alias resolved; skill counted as matched	Pass

ID	Test Case	Expected Result	Status
UT04	Job with no required skills	Treated as a 100% skill match	Pass
UT05	Resume with no years of experience mentioned	Experience fit is null (neutral), weight redistributed	Pass
UT06	Skill declared on candidate profile but not in resume	Included in the skill match	Pass
UT07	"X years of experience" phrasing	Correct number of years extracted	Pass
UT08	"experience of X years" phrasing	Correct number of years extracted	Pass
UT09	"X+ years in" phrasing	Correct number of years extracted	Pass
UT10	Multiple year figures in one resume	Maximum figure returned	Pass
UT11	No years / empty input	null returned	Pass
UT12	Detect PhD, Master's and Bachelor's degrees	Correct education level	Pass
UT13	Several education levels mentioned	Highest level returned	Pass
UT14	Dictionary skills present in text	Skills detected	Pass
UT15	Substring such as "java" inside "javascript"	No false-positive match	Pass
UT16	Unknown term passed to alias resolver	Returned unchanged (lowercased)	Pass

8.3 Unit Test Cases — Security, Notifications and LLM Layer

Table 8.3 Unit test cases for middleware, mailer and semantic layer

ID	Test Case	Expected Result	Status
UT17	Request without Authorization header	HTTP 401	Pass
UT18	Invalid JWT token	HTTP 401	Pass
UT19	Valid token and active user	req.user attached, next() called	Pass
UT20	Valid token but deactivated user	HTTP 403	Pass
UT21	Token for a user that no longer exists	HTTP 401	Pass
UT22	User role allowed / not allowed by authorize()	next() called / HTTP 403	Pass
UT23	SMTP not configured	Falls back to console-logging transport	Pass
UT24	SMTP configured	Real SMTP transport used	Pass
UT25	Email transport failure	Error logged, never thrown	Pass
UT26	LLM API key not configured	Returns null without any network call	Pass
UT27	Valid JSON response from LLM	Parsed into score and summary	Pass
UT28	LLM call fails or returns non-JSON	Returns null, does not throw	Pass

8.4 Integration Test Cases — REST API

Table 8.4 Integration test cases

ID	Test Case	Expected Result	Status
IT01	Register an employer / a candidate	HTTP 201 with token	TBD
IT02	Self-registration as admin	Rejected	TBD
IT03	Duplicate registration	Rejected	TBD
IT04	Login with correct / wrong password	Token returned / HTTP 401	TBD
IT05	Protected request without token	HTTP 401	TBD
IT06	Candidate tries to post a job	HTTP 403	TBD
IT07	Employer posts a job with required skills	HTTP 201, skills stored	TBD
IT08	List open jobs without authentication	Job listed publicly	TBD
IT09	Employer tries to apply to a job	HTTP 403	TBD
IT10	Candidate applies with a resume	HTTP 201 with AI match score and matched skills	TBD
IT11	Candidate applies twice to the same job	HTTP 409	TBD
IT12	Candidate lists own applications	Application returned	TBD
IT13	Owning employer views ranked applicants	Applicants sorted by score	TBD
IT14	Different employer views applicants	HTTP 403	TBD
IT15	Owning employer updates status / invalid status	Updated / HTTP 400	TBD
IT16	Non-admin calls admin API	HTTP 403	TBD
IT17	Admin views platform statistics	Statistics returned	TBD
IT18	Admin deactivates a user	User can no longer log in	TBD
IT19	Admin force-closes a job	Job status becomes closed	TBD

8.5 Frontend Component Test Cases

Table 8.5 Frontend test cases

ID	Component / Page	Test Case	Status
FT01	MatchScoreBadge	Renders percentage; green for ≥ 75, amber for 50–74, red for < 50	Pass
FT02	JobCard	Renders title, company, location; links to details; formats job type; shows at most 6 skills	Pass
FT03	PrivateRoute	Shows loading state; redirects to /login with no user; redirects to / for wrong role; renders content for correct role	Pass

ID	Component / Page	Test Case	Status
FT04	AuthContext	No user without token; loads user with stored token; clears rejected token; login and logout	Pass
FT05	Login page	Candidates navigate to /jobs, employers to /employer/jobs; error shown on failed login	Pass

8.6 Validation of the Matching Engine

Beyond automated tests, the matching engine was validated with the four sample candidates described in Section 7.11. The engine's ranking agreed with the expected human ranking in every position, scores were spread over a wide range (0 to 84), and the matched/missing skill lists correctly explained each result. Alias resolution was confirmed by the fact that "React", "Node.js", "Express" and "REST API" written in the resumes were matched to the canonical forms react.js, node.js, express.js and rest.

8.7 Issues Identified and Resolved

Table 8.6 Issues handled during development

Issue	Resolution
Candidates whose resumes did not state years of experience were unfairly penalised.	Experience fit returns null when unknown and its weight is redistributed to skill match and text similarity.
Abbreviations such as "js" or "k8s" were not matched with job requirements.	Introduced a skill alias table and canonicalisation of both job and resume skills.
Short skill names were falsely detected inside longer words (e.g. "java" in "javascript").	Word-boundary aware skill detection.
Uploaded files remained on disk when an application failed.	Uploaded files are deleted on every failure path (job not found, closed job, duplicate, parse error, server error).
A candidate could apply to the same job more than once.	Application-level check returning HTTP 409 plus a unique compound database index on (job, candidate).
Deactivated users could continue using an existing token.	protect middleware checks the isActive flag on every request.
Resumes containing personal data could be accessed by URL.	Uploads are not served as static files; access only through authorised endpoints.
Email or LLM service failures could block an application.	Both services are wrapped so that failures are logged and never thrown.

8.8 Test Results Summary

Table 8.7 Summary of automated test execution

Suite	Framework	Tests	Result
Backend unit tests (6 suites)	Jest	37	37 passed
Frontend component tests (5 files)	Vitest + React Testing Library	20	20 passed
Backend API integration tests	Jest + Supertest + mongodb-memory-server	21	TBD

All 57 unit and component tests passed. The results confirm that the AI engine, security controls, notification logic and user-interface components behave as specified, and that failures in optional services degrade gracefully without affecting the core recruitment workflow.

CHAPTER IX
CONCLUSION

This project set out to reduce the effort, cost and inconsistency of resume screening by building a full-stack job portal with integrated AI-based resume screening and candidate matching. The objectives defined at the start of the project have been achieved.

9.1 Objectives Achieved

- A complete, responsive job portal was developed on the MERN stack with separate experiences for candidates, employers and administrators.
- Secure JWT-based authentication, bcrypt password hashing and role- and ownership-based authorisation were implemented.
- Resumes in PDF, DOCX, DOC and TXT formats are parsed automatically.
- An explainable AI matching engine scores every application from 0 to 100 using skill overlap, TF cosine text similarity and experience fit, with alias-aware skill matching and fair handling of missing information.
- Employers receive a ranked applicant list with detailed reasons for each score and can manage the hiring pipeline.
- An administrator dashboard, automatic email notifications and an optional LLM-based semantic layer were added.
- A recruitment dataset of 12,000 records was analysed and a Random Forest classification pipeline was built.
- The system was verified with 57 passing unit and component tests and a 21-case API integration suite.

9.2 Key Learnings

- Designing and building a production-style, three-tier web application with a REST API and a NoSQL database.
- Applying information-retrieval and NLP techniques (tokenisation, stop-word removal, term-frequency vectors, cosine similarity) to a real business problem.
- The importance of explainability and fairness when AI is used in decisions that affect people, such as hiring.

- Building secure systems that protect personal data, and designing for graceful degradation when external services fail.
- The value of automated testing for maintaining confidence while adding features.
- Translating HR business requirements such as time-to-hire and candidate experience into technical features.

9.3 Benefits and Impact

For organisations, JobMatch AI shortens the initial screening stage, standardises evaluation criteria and lets recruiters focus on high-value activities such as interviewing. Because it is built on open-source technology and does not require paid AI services, it is affordable for small and medium enterprises, start-ups and campus placement cells. For candidates, it provides a simple application process, instant feedback in the form of a match score and timely status updates. Overall, the project demonstrates that practical, transparent AI can be embedded into everyday business processes to improve efficiency and decision quality, while keeping the final decision in human hands.

CHAPTER X
FUTURE ENHANCEMENTS

Although the current system meets its objectives, several enhancements can increase its accuracy, reach and business value:

1. **Semantic embeddings:** replace or complement term-frequency similarity with sentence-embedding models (for example transformer-based embeddings) and a vector database, so that meaning rather than word overlap drives similarity.
2. **Learning from hiring outcomes:** use the statuses recorded in the platform (shortlisted, rejected, hired) as labels to train a supervised model, building on the Random Forest pipeline developed in this project, and serve it as a scoring microservice.
3. **Automatic skill discovery:** extract skills with named-entity recognition and skill taxonomies (such as ESCO or O*NET) instead of a fixed dictionary.
4. **Experience from dates:** calculate total experience from employment date ranges in the resume.
5. **OCR support:** read scanned or image-based resumes using optical character recognition.
6. **Bias monitoring and fairness audits:** add reports that compare score distributions across groups and allow administrators to tune weights.
7. **Job recommendations for candidates:** reverse the matching to recommend the most suitable open jobs to each candidate.
8. **Interview scheduling and collaboration:** calendar integration, interview feedback forms and team comments on candidates.
9. **Recruitment analytics dashboard:** time-to-hire, funnel conversion rates, source effectiveness and skill-gap analysis for management.
10. **Cloud deployment and scalability:** containerisation with Docker, cloud object storage for resumes, CI/CD pipelines and horizontal scaling.
11. **Integrations:** import profiles from professional networks, post jobs to external job boards and connect with HRMS systems.
12. **Mobile application and multilingual support** to reach a wider audience.

CHAPTER XI
REFERENCES

Anthropic. *Claude API Documentation*. Anthropic, 2026. https://docs.anthropic.com/

Breiman, Leo. "Random Forests." *Machine Learning* 45.1 (2001): 5–32.

Dastin, Jeffrey. "Amazon Scraps Secret AI Recruiting Tool That Showed Bias Against Women." *Reuters*, 10 Oct. 2018.

Dessler, Gary. *Human Resource Management*. 16th ed., Pearson, 2020.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." *Proceedings of NAACL-HLT* (2019): 4171–4186.

Express.js Team. *Multer — Node.js Middleware for Handling multipart/form-data*. 2026. https://github.com/expressjs/multer

Faliagka, Evanthia, Kostas Ramantas, Athanasios Tsakalidis, and Giannis Tzimas. "Application of Machine Learning Algorithms to an Online Recruitment System." *Proceedings of the Seventh International Conference on Internet and Web Applications and Services* (2012): 215–220.

Government of India. *The Digital Personal Data Protection Act, 2023*. Ministry of Law and Justice, 2023.

Jones, Michael, John Bradley, and Nat Sakimura. *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force, 2015.

Jurafsky, Daniel, and James H. Martin. *Speech and Language Processing*. 3rd ed. draft, Stanford University, 2023.

Kenthapadi, Krishnaram, Benjamin Le, and Ganesh Venkataraman. "Personalized Job Recommendation System at LinkedIn: Practical Challenges and Lessons Learned." *Proceedings of the Eleventh ACM Conference on Recommender Systems* (2017): 346–347.

Malinowski, Jochen, Tobias Keim, Oliver Wendt, and Tim Weitzel. "Matching People and Jobs: A Bilateral Recommendation Approach." *Proceedings of the 39th Hawaii International Conference on System Sciences* (2006).

Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

Meta Platforms. *React Documentation*. 2026. https://react.dev/

MongoDB Inc. *MongoDB Manual*. 2026. https://www.mongodb.com/docs/manual/

Mongoose. *Mongoose ODM Documentation*. 2026. https://mongoosejs.com/docs/

Nodemailer. *Nodemailer Documentation*. 2026. https://nodemailer.com/

OpenJS Foundation. *Express — Node.js Web Application Framework*. 2026. https://expressjs.com/

OpenJS Foundation. *Jest Documentation*. 2026. https://jestjs.io/

OpenJS Foundation. *Node.js Documentation*. 2026. https://nodejs.org/docs/

Paparrizos, Ioannis, B. Barla Cambazoglu, and Aristides Gionis. "Machine Learned Job Recommendation." *Proceedings of the Fifth ACM Conference on Recommender Systems* (2011): 325–328.

Pedregosa, Fabian, et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research* 12 (2011): 2825–2830.

Pressman, Roger S., and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 9th ed., McGraw-Hill Education, 2019.

Provos, Niels, and David Mazières. "A Future-Adaptable Password Scheme." *Proceedings of the USENIX Annual Technical Conference* (1999).

Raghavan, Manish, Solon Barocas, Jon Kleinberg, and Karen Levy. "Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices." *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency* (2020): 469–481.

Roy, Pradeep Kumar, Sarabjeet Singh Chowdhary, and Rocky Bhatia. "A Machine Learning Approach for Automation of Resume Recommendation System." *Procedia Computer Science* 167 (2020): 2318–2327.

Salton, Gerard, and Christopher Buckley. "Term-Weighting Approaches in Automatic Text Retrieval." *Information Processing & Management* 24.5 (1988): 513–523.

scikit-learn Developers. *scikit-learn User Guide*. 2026. https://scikit-learn.org/stable/user_guide.html

Siting, Zheng, Hong Wenxing, Zhang Ning, and Yang Fan. "Job Recommender Systems: A Survey." *Proceedings of the 7th International Conference on Computer Science & Education* (2012): 920–924.

Sommerville, Ian. *Software Engineering*. 10th ed., Pearson, 2015.

Tailwind Labs. *Tailwind CSS Documentation*. 2026. https://tailwindcss.com/docs

Testing Library. *React Testing Library Documentation*. 2026. https://testing-library.com/docs/react-testing-library/intro/

Vite Team. *Vite Documentation*. 2026. https://vitejs.dev/

Vitest Team. *Vitest Documentation*. 2026. https://vitest.dev/

CHAPTER XII
APPENDIX

Appendix A — Questionnaire

The following questionnaire is designed to evaluate the usefulness and acceptance of the JobMatch AI system among recruiters and job seekers. Part A collects respondent details; Parts B and C use a five-point Likert scale (Strongly Disagree (1) / Disagree (2) / Neutral (3) / Agree (4) / Strongly Agree (5)).

Part A — Respondent Profile

1. Name (optional): _____
2. Role: ☐ Recruiter / HR professional ☐ Hiring manager ☐ Job seeker ☐ Student
3. Organisation type: ☐ Start-up ☐ Small/medium enterprise ☐ Large enterprise ☐ Educational institution ☐ Not applicable
4. Experience in recruitment or job searching: ☐ Less than 1 year ☐ 1–3 years ☐ 3–5 years ☐ More than 5 years
5. How many applications do you typically review (or submit) for one job opening? ☐ Fewer than 25 ☐ 25–100 ☐ 100–500 ☐ More than 500
6. Have you used an applicant tracking system or AI-based screening tool before? ☐ Yes ☐ No

Table 12.1 Part B — Recruiter / employer perception

No.	Statement	1	2	3	4	5
B1	Manual screening of resumes takes a significant amount of my time.					
B2	The AI match score helped me identify suitable candidates quickly.					
B3	The ranking of applicants matched my own judgement of their suitability.					
B4	The list of matched and missing skills made the score easy to understand.					
B5	Recognising synonyms and abbreviations (e.g. JS / JavaScript) improves screening.					
B6	Not scoring education level makes the screening fairer.					
B7	The system would reduce the time-to-hire in my organisation.					
B8	Posting jobs and managing applicant status was easy.					

No.	Statement	1	2	3	4	5
B9	I trust the system to support, but not replace, my hiring decisions.					
B10	I would recommend this system to other recruiters.					

Table 12.2 Part C — Job seeker perception

No.	Statement	1	2	3	4	5
C1	Searching and filtering jobs on the portal was easy.					
C2	Applying with a resume upload was simple and quick.					
C3	Seeing my match score immediately after applying was useful.					
C4	Email notifications kept me informed about my application status.					
C5	I feel AI-based screening evaluates candidates more consistently than manual screening.					
C6	I am comfortable with my resume being screened by an AI system.					
C7	I would use this portal again to apply for jobs.					

Part D — Open-ended Questions

1. What did you find most useful about the system?
2. What difficulties, if any, did you face while using the system?
3. What additional features would you like to see in future versions?

Appendix B — Glossary

Table 12.3 Glossary of terms

Term	Meaning
ATS	Applicant Tracking System — software that manages job applications
API / REST	Application Programming Interface; Representational State Transfer, an architectural style for web services
JWT	JSON Web Token — a signed token used for stateless authentication
MERN	MongoDB, Express, React and Node.js technology stack
NLP	Natural Language Processing — techniques that allow computers to process human language
TF	Term Frequency — the number of times a word occurs in a document
Cosine similarity	A measure of similarity between two vectors based on the angle between them
LLM	Large Language Model — an AI model trained on large text corpora (e.g. Claude)

Term	Meaning
Random Forest	An ensemble machine-learning method that combines many decision trees
Time-to-hire	Number of days between a candidate applying and accepting an offer

Appendix C — Project Folder Structure

```
backend/
  config/db.js              MongoDB connection
  models/                   User, Job, Application schemas
  middleware/               auth (JWT + active check), role guard, multer
upload, error handler
  utils/
    resumeParser.js         PDF/DOCX/TXT -> plain text extraction
    skillsDictionary.js     curated skill keyword list + extractor
    skillAliases.js         skill synonym/alias table
    experienceParser.js     years-of-experience + education level extraction
    aiMatcher.js            local AI screening/matching engine
    semanticMatcher.js      optional LLM-based semantic scoring layer
    mailer.js               email notifications (SMTP or console fallback)
    seed.js                 script to create the first admin account
  controllers/, routes/     REST API (auth, users, jobs, applications, admin)
  tests/unit/               Jest unit tests
  tests/integration/        Jest + Supertest API tests (in-memory MongoDB)
  server.js                 application entry point
frontend/
  src/api/axios.js          API client (attaches JWT)
  src/context/AuthContext.jsx
  src/components/           Navbar, JobCard, JobForm, MatchScoreBadge,
PrivateRoute
  src/pages/                Home, Login, Register, Jobs, JobDetails, PostJob,
EditJob,
                            EmployerJobs, Applicants, CandidateApplications,
Profile,
                            AdminDashboard, NotFound
  src/test/                 Vitest + React Testing Library tests
AI_Job_Assistance.ipynb     Recruitment data analysis and Random Forest model
```

Appendix D — Installation and User Manual

D.1 Prerequisites

- Node.js 18 or later
- A running MongoDB instance (local or MongoDB Atlas)

D.2 Backend Setup

```
cd backend
cp .env.example .env      # set MONGO_URI and JWT_SECRET
npm install
npm run dev               # starts on http://localhost:5000

# create the first administrator account
ADMIN_EMAIL=admin@example.com ADMIN_PASSWORD=change_me npm run seed:admin
```

D.3 Frontend Setup

```
cd frontend
cp .env.example .env      # VITE_API_URL=http://localhost:5000/api
npm install
npm run dev               # starts on http://localhost:5173
```

D.4 Using the Application

1. Open http://localhost:5173 in a web browser.
2. Register as an **employer**, post a job and list its required skills.
3. In another browser window, register as a **candidate**, update the profile skills, browse jobs and apply with a resume. The AI match score is shown immediately.
4. As the employer, open **My Postings → View Applicants** to see the ranked list, expand AI match details and update statuses.
5. Log in with the administrator account and open **/admin** for platform statistics and moderation.

D.5 Running the Tests

```
cd backend  && npm test     # Jest: unit + API integration tests
cd frontend && npm test     # Vitest: component and page tests
```

Appendix E — Environment Variables

Table 12.4 Backend configuration variables

Variable	Purpose
PORT	API server port (default 5000)
MONGO_URI	MongoDB connection string
JWT_SECRET	Secret used to sign authentication tokens
CLIENT_URL	Allowed frontend origin for CORS
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_FROM, SMTP_SECURE	Optional email server settings; if absent, emails are logged to the console
ANTHROPIC_API_KEY	Optional; enables the LLM semantic scoring layer

Appendix F — Sample API Response (Application with AI Match Details)

```
{
  "application": {
    "_id": "...",
    "job": "...",
    "candidate": "...",
    "resumeOriginalName": "Arjun_Resume.pdf",
    "matchScore": 84,
    "matchDetails": {
```

```
      "skillMatchPercent": 100,
      "textSimilarityPercent": 34,
      "experienceFitPercent": 100,
      "candidateYearsOfExperience": 4,
      "educationLevel": "bachelor",
      "matchedSkills": ["javascript", "react.js", "node.js", "express.js",
                        "mongodb", "rest", "git"],
      "missingSkills": [],
      "semanticScore": null,
      "semanticSummary": null
    },
    "status": "applied"
  }
}
```

Appendix G — Skills Dictionary Categories

Table 12.5 Categories of the curated skills dictionary

Category	Examples
Languages	JavaScript, TypeScript, Python, Java, C++, C#, Go, Rust, PHP, Kotlin, SQL, HTML, CSS
Frontend	React, Redux, Vue, Angular, Next.js, Svelte, Tailwind, Bootstrap, Vite
Backend	Node.js, Express, Django, Flask, FastAPI, Spring Boot, .NET, Laravel, GraphQL, REST, Microservices
Databases	MongoDB, MySQL, PostgreSQL, Redis, Elasticsearch, DynamoDB, Firebase, Oracle
Cloud / DevOps	AWS, Azure, GCP, Docker, Kubernetes, Terraform, Jenkins, CI/CD, Linux, Git
Data / AI	Machine Learning, Deep Learning, NLP, Computer Vision, TensorFlow, PyTorch, scikit-learn, pandas, Spark, Tableau, Power BI, LLM
Mobile	Android, iOS, React Native, Flutter
Testing	Jest, Mocha, Cypress, Selenium, JUnit, PyTest, Test Automation
Management	Agile, Scrum, Kanban, Jira, Project Management, Leadership, Stakeholder Management
Design / Security	Figma, UI/UX, Photoshop, Illustrator; Cybersecurity, OAuth, JWT

Appendix H — Test Execution Log (Excerpt)

```
$ cd backend && npx jest tests/unit
PASS tests/unit/aiMatcher.test.js
PASS tests/unit/auth.middleware.test.js
PASS tests/unit/experienceParser.test.js
PASS tests/unit/mailer.test.js
PASS tests/unit/semanticMatcher.test.js
PASS tests/unit/skillsDictionary.test.js
Test Suites: 6 passed, 6 total
Tests:       37 passed, 37 total

$ cd frontend && npx vitest run
 ✓ src/test/components/PrivateRoute.test.jsx (4 tests)
 ✓ src/test/context/AuthContext.test.jsx (5 tests)
 ✓ src/test/pages/Login.test.jsx (3 tests)
```

```
✓ src/test/components/JobCard.test.jsx (4 tests)
✓ src/test/components/MatchScoreBadge.test.jsx (4 tests)
Test Files  5 passed (5)
     Tests  20 passed (20)
```

Appendix I — Plagiarism Report

[Attach the plagiarism report generated through the university-approved tool here.]